MICRO PROJECT REPORT · SEMESTER 5 · FET

PADDOCK

Formula 1 Archive

SUBMITTED BY

Jugal Patel — 24162581032

Krish Patel — 24162581017

Paavan Sharma — 24162581052

A complete Formula 1 archive covering every season from 2000 to 2026, built as an Angular single-page application with a bundled 1.3 MB dataset, a full OpenF1 live-timing layer and a brushed-silver editorial design system.

DATE
27 October 2026

DATA THROUGH
26 October 2026

STACK
Angular 21 · Material 3 · OpenF1

DOCUMENT
Code, planning and interface report

Project at a glance

The numbers, the scope and the three people who built it.

27	526	10,954	11
SEASONS BUNDLED	RACES INDEXED	RACE RESULTS	SCREENS SHIPPED

Table 1 — Project identification

ITEM	DETAIL
Report title	Paddock — Formula 1 Archive: code, planning and interface
Subject / semester	FET · Semester 5
Students	Jugal Patel (24162581032) · Krish Patel (24162581017) · Paavan Sharma (24162581052)
Archive scope	2000–2026, reconciled to 26 October 2026
Live-data scope	OpenF1, 2023 to the present; replay-first, optional paid token
Frontend	Angular 21.2 standalone · signals · Angular Material 21 · SCSS
Data	9 CSV files + meta.json · PapaParse · RxJS · Jolpica and OpenF1 REST
Build	Angular CLI 21 / esbuild · 580.38 kB raw / 154.23 kB transferred
Document	Generated from the live source tree; code listings are read from source
Date	27 October 2026

This document is written to be read alongside the repository. It explains what the application does, how the data reaches the screen, why the interface looks the way it does, and how the finished build was verified. Every code listing is sliced from the working tree at build time, so the report cannot quietly drift away from the implementation it describes.

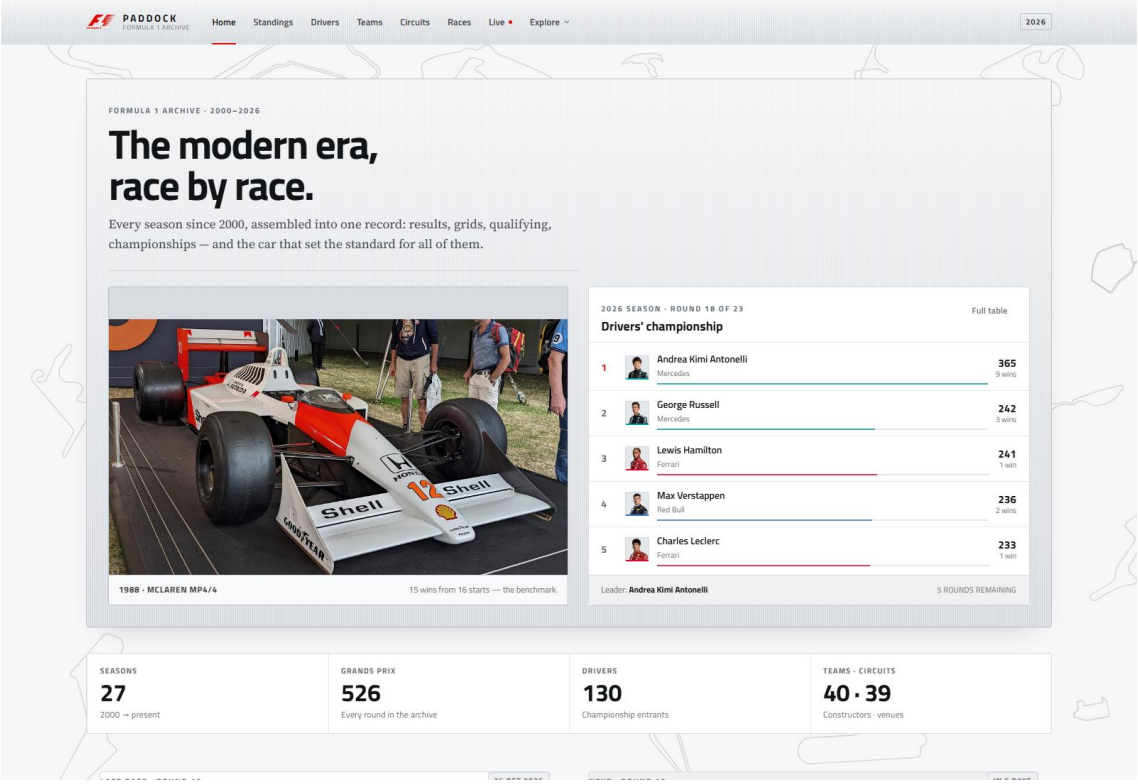


Figure 1 — The home screen at 1600 × 1100, the entry point for the whole archive.

How this report is organised

- **Chapters 1–3** set out the problem, the planning method, the decisions that were hard to reverse and the requirements behind the eleven routes.
- **Chapters 4–6** describe the architecture, the two data pipelines and the page-by-page implementation. The code listings in these chapters are read from the working tree at build time.
- **Chapters 7–8** cover the design system, the background algorithm and the imagery licensing rules.
- **Chapters 9–13** present the evidence: build and route verification, all eleven screenshots, performance and accessibility, limitations, and the conclusion.
- **Appendices A–C** map the source tree, list every build script and give the full rebuild sequence.

The companion artefacts — `README.docx` / `.pdf` and `f1-presentation.pptx` / `.pdf` — are generated from the same repository. Where this report cites a count or a file, the value comes from `meta.json`, the build log or the source tree rather than from memory.

Contents

Chapters 1–13, references and appendices. Page numbers are refreshed when the PDF is exported.

Abstract 7
1. Introduction 8
1.1 The problem 8
1.2 Objectives 8
1.3 Scope and constraints 8
2. Planning and methodology 9
2.1 Working method 9
2.2 Phases 9
2.3 Decision log 10
2.4 The background, three times 11
2.5 Risks and mitigations 12
3. Requirements 13
3.1 Functional requirements 13
3.2 Non-functional requirements 13
3.3 Deliberately out of scope 14
4. System architecture 15
4.1 Overview 15
4.2 Application shell 15
4.3 Routing and lazy loading 16
4.4 Services and state 16
4.5 Directory and shared library 17
4.6 Optional legacy backend 17
5. Data pipeline 18
5.1 Sources and scope 18
5.2 Build-time ingest 18
5.3 Runtime indexing 19
5.4 Derived views 20
5.5 OpenF1 integration 21
5.6 Changing the scope 25

6. Frontend implementation 26
6.1 Page composition 26
6.2 Home 26
6.3 Standings 27
6.4 Drivers and teams 27
6.5 Circuits and races 28
6.6 Race detail 28
6.7 Live timing 28
6.8 History 29
6.9 Rivalry 29
6.10 Credits 29
6.11 Shared components and conventions 30
7. Design system 31
7.1 Tokens 31
7.2 Typography 31
7.3 Surfaces and elevation 32
7.4 Red as signal 32
7.5 The background algorithm 32
7.6 Motion 34
7.7 Responsive behaviour 34
8. Imagery and licensing 35
8.1 The asset manifest 35
8.2 Why the archive is local 35
8.3 Credits and trademark position 35
9. Verification and testing 37
9.1 Build health 37
9.2 Route smoke tests 37
9.3 Visual measurement 38
9.4 Screenshot capture 38
9.5 Document verification 38
10. Screenshots 39
10.1 Archive screens 39
10.2 Live, long-form and credits 43

11. Performance and accessibility 45
11.1 Bundle and loading 45
11.2 Accessibility 45
11.3 Data-scale performance 46
12. Limitations and future work 47
12.1 Data limitations 47
12.2 Technical limitations 47
12.3 Next steps 47
13. Conclusion 49
References 50
Appendix A — Source map 51
Appendix B — Build and data scripts 53
Appendix C — Rebuilding the project 54

A note on the numbers. Counts in this report come from `frontend/src/assets/archive/meta.json` and the production build log. The archive is current to 26 October 2026; the 2026 season is still in progress, so the application names a leader rather than a champion.

Abstract

Paddock is a single-page Formula 1 archive that puts twenty-seven seasons of racing data, thirty circuit outlines and a live telemetry viewer behind one brushed-silver interface.

The application covers every season from 2000 to 2026. A pruned 1.3 MB dataset — 130 drivers, 40 constructors, 39 circuits, 526 races, 10,954 race results and 9,968 qualifying rows — is bundled with the frontend, parsed once at start-up and indexed in the browser. That makes career totals, championship tables, circuit histories and head-to-head comparisons instant, and it keeps the archive usable offline. From 2023 onward the same interface adds OpenF1 sessions, race control messages, weather and per-lap car telemetry.

The interface is deliberately not another dark, glassy dashboard. It is an editorial system built on matte silver surfaces, hairline rules, tabular numerals and one red accent. Thirty distinct circuit maps, drawn as black outlines at a uniform 20% ink, form a full-bleed background field that rebuilds itself for tablet and phone widths. Red is reserved for signal: the F1 mark, the active navigation rule, a winning row and one primary action per view.

The report documents the whole build: the planning phases and rejected alternatives, the functional and non-functional requirements, the Angular architecture, the two data pipelines, the page-by-page implementation, the design system, image licensing, verification, performance and known limitations. Code listings are extracted from the source tree when this document is generated, so they show the code that actually shipped. The finished production build is 580.38 kB raw and 154.23 kB transferred, with every non-home route lazy-loaded.

> **AT A GLANCE**
> 27 seasons · 526 races · 10,954 results · 11 screens · 30 unique background maps · 580.38 kB initial bundle · 11-route smoke test passing · data reconciled to 26 October 2026.

1. Introduction

Formula 1 is one of the most thoroughly documented sports in the world, yet its history is still split across statistics sites, image galleries, PDFs and paywalled timing services.

1.1 The problem

A reader who wants to compare two drivers usually has to open several sites: one for career totals, another for season-by-season standings, a third for race classification and a fourth for circuit layouts. Live timing is separated from history altogether. The result is a fragmented reading experience in which the same sport looks like four unrelated products.

The project asks a narrower question: can one interface hold the archive and the live session without turning into a dashboard? The answer adopted here is a static, bundled archive that behaves like a reference book, with a live layer that appears only when the reader asks for it.

1.2 Objectives

1. **Cover the whole modern era.** Every season from 2000 to the current season, including the 2026 calendar and the new Audi, Cadillac and Madring entries.
2. **Answer archive questions instantly.** Career totals, titles, team histories, circuit records, season standings and race classification should not require a network round trip.
3. **Make live data optional, not essential.** OpenF1 sessions, race control and telemetry should work as a replay without a token, and as a live poller when a paid token is supplied.
4. **Build a coherent visual system.** One silver theme, one red accent, consistent panels and tables, and a background that belongs to the sport rather than generic decoration.
5. **Ship a small, accessible, static bundle.** No server is required to run the application, and every route must remain usable on a phone and with reduced motion enabled.

1.3 Scope and constraints

The archive window is fixed at **2000–2026**, reconciled to **26 October 2026**. Ergast supplies the 2000–2024 base; Jolpica supplies 2025 and 2026. OpenF1 supplies live and historical session data from 2023 onward. The app is a student project: it is unofficial, non-commercial, and every trademark and image source is acknowledged on the credits screen.

Two constraints shaped most of the engineering. First, the 2026 season is unfinished — 18 of 23 rounds were complete when the archive was last synced — so the UI must never present the current leader as a champion. Second, OpenF1's free tier is rate-limited to three requests per second, so the live client needs a token bucket and an aggressive response cache rather than naive polling.

2. Planning and methodology

The project was built in vertical slices: lock the data, ship one complete screen at a time, then harden the shared system once the patterns were visible.

2.1 Working method

The repository began as a Node/Express and MongoDB experiment. Early on it became clear that the database added deployment friction without improving the reading experience: the dataset is small enough to ship, and every archive view is read-only. The frontend was therefore rebuilt as a standalone Angular application, and the Express API was left in place as an optional reference implementation that the UI does not call.

Work proceeded in short loops. Each loop started from a real question — "what does a career page need to show?", "how does a cancelled race appear?" — then added the smallest service method and the smallest screen that answered it. Shared components were extracted only after two or three pages had asked for the same thing, which kept the component library small and prevented speculative abstraction.

Verification was treated as part of the loop rather than a final phase. The production build was run after each visual change, the eleven routes were loaded in a headless browser and checked for expected content, and the background layer was measured from screenshots rather than judged by eye alone.

2.2 Phases

The schedule below is indicative rather than a dated log; it records the order in which the work actually depended on itself. The two red bars are the phases that produced the most rework: the archive screens (because the shared table and panel patterns were still forming) and verification (because every earlier decision was re-checked there).

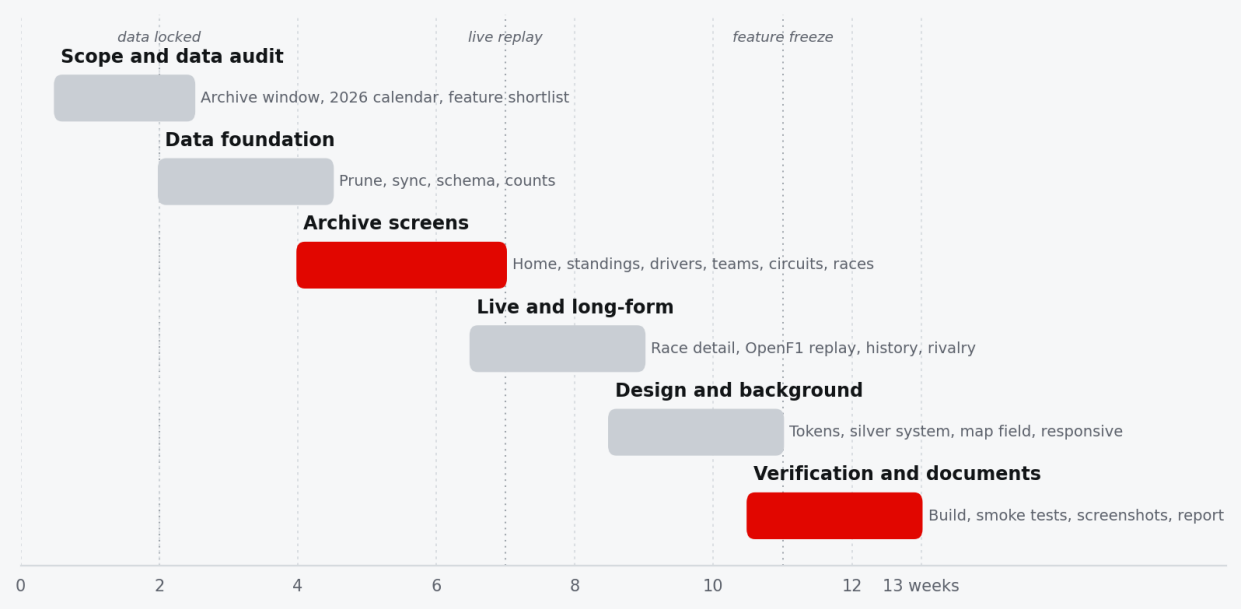


Figure 2 — Indicative build sequence. Data had to be locked before the archive screens could be finished, and the design system was hardened only after the pages exposed its gaps.

Table 2 — Build phases and deliverables

PHASE	FOCUS	OUTPUT
1 · Scope	Archive window, 2026 calendar, feature shortlist	Route list, data dictionary, decision to drop the runtime database
2 · Data foundation	Prune Ergast, append Jolpica, rebuild standings	Nine CSVs plus `meta.json`, 27 seasons, reproducible `data:build`
3 · Archive screens	Home, standings, drivers, teams, circuits, races	Six working screens and the first shared panel, table and header primitives
4 · Live and long-form	Race detail, OpenF1 replay, history, rivalry	Session picker, timing tower, telemetry trace, article and comparison engine
5 · Design and background	Tokens, typography, silver surfaces, map field	One theme, responsive rules, 30-map background, reduced-motion handling
6 · Verification and documents	Build, route smoke tests, screenshots, report	Production bundle, eleven captures, README, deck and this report

2.3 Decision log

The most useful planning artefact was not the schedule but the list of decisions that could not be reversed cheaply. Each one below closed off a plausible alternative, so it is recorded with the reason rather than just the outcome.

Table 3 — Decisions that constrained the rest of the build

DECISION	ALTERNATIVE REJECTED	WHY
Ship the archive as static CSV	Keep MongoDB behind an Express API	1.3 MB is small enough to bundle; static hosting removes a runtime dependency and makes every archive view work offline.
One silver editorial theme	Dark glass dashboard, theme toggle	The subject is archival. A single light system reads like a printed reference and avoids maintaining two colour languages.

DECISION	ALTERNATIVE REJECTED	WHY
One red accent used as signal	Team colours as page themes	Forty team palettes would fragment the interface. Team colour survives in badges, bars and two-pixel rules only.
Thirty-map full-bleed field	Corner collage with a radial mask	The corner version measured a 15:1 density imbalance and made the top-left feel empty. The field is now balanced at roughly 1:1.
Replay-first live page	Live-only timing that requires a token	OpenF1's free tier covers history. Replay keeps the page useful to every reader; polling is opt-in.
Hand-built SVG charts	A charting dependency	Two chart types did not justify a library. Hand-built SVG keeps the bundle small and the styling consistent with the tokens.
Lazy-load every route except Home	One eager bundle	The live and history screens carry most of the code. Splitting them keeps the first paint under 155 kB transferred.
No champion until the season is complete	Show the current leader as champion	The archive is current to 26 October 2026 and 2026 is unfinished. The data model carries a `completeYears` set so the UI can tell the difference.

2.4 The background, three times

The background went through three distinct designs. The first was a twelve-track collage piled into the bottom-right corner. It was atmospheric but unbalanced: the top-left of every page was empty, and the corner became visually noisy behind the tables.

The second version raised the count to twenty-four, varied opacity between 6% and 19% and added a radial mask anchored to the corner. Measurement showed the problem had not gone away: at 1600 px the ink centroid sat at **71% / 67%**, the bottom-right carried roughly **15 times** the ink of the top-left, and total coverage was only **1.52%**. The mask was hiding the imbalance rather than fixing it.

The third version removed the mask entirely and rebuilt the layer as a field. Thirty distinct circuits — every generated outline except Buddh — sit on a loose 6×5 staggered grid, all at the same 20% ink, with sizes between 125 and 195 px and rotations between −35° and +35°. Measured at 1600 px, the centroid moved to **49% / 51%**, the two halves became roughly equal, and coverage rose to **2.51%**. The field rebuilds to twenty maps at 1400 px and ten at 760 px instead of cropping, so the edges stay covered at every width.

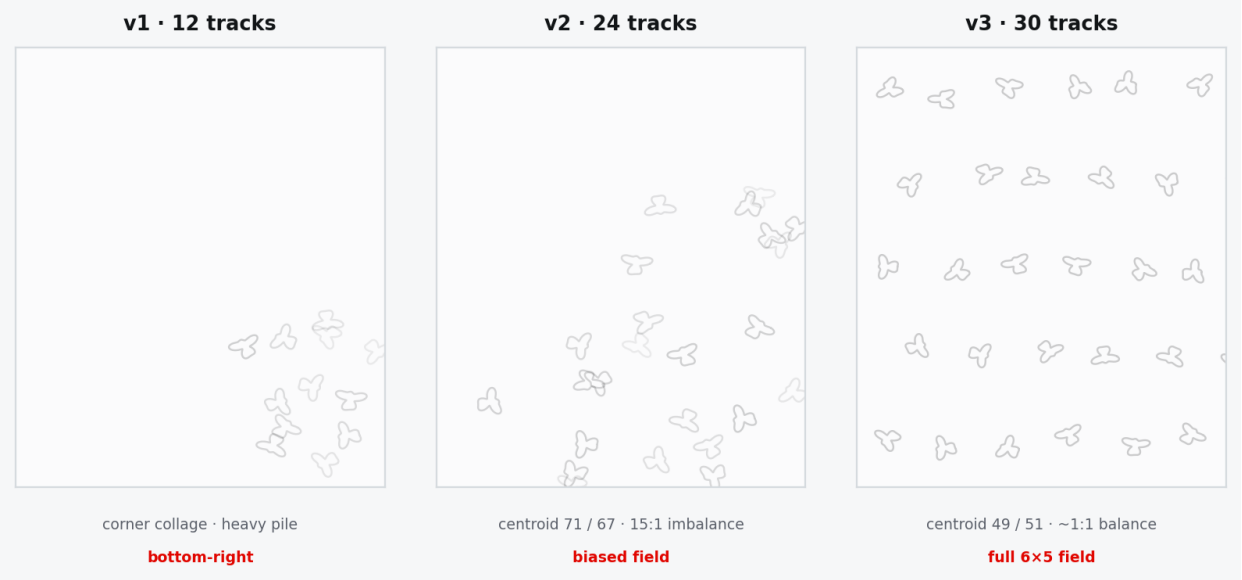


Figure 3 — The three background systems. The measured centroid moved from 71/67 in the masked corner version to 49/51 in the full field; the third panel is the design that shipped.

DESIGN RULE

A background should be measurable. If the brief says “spread across the viewport”, the build needs a number that can fail — in this case the ink centroid and the quadrant ratio.

2.5 Risks and mitigations

Table 4 — Principal risks

RISK	IMPACT	MITIGATION
OpenF1 rate limit (3 req/s)	Live board stalls or errors	Token bucket with burst 3 and a 340 ms refill, plus an in-memory response cache for immutable historical data.
2026 calendar changes mid-season	Wrong winners, phantom rounds	A race counts as complete only when it has results; cancelled rounds keep their calendar entry but never produce a classification.
Incomplete portrait coverage	Broken images on older drivers	The asset manifest lists only files that exist; components fall back to monograms and typographic crests.
Bundle growth	Slow first paint	Route-level splitting, a 1.5 MB warning budget and a build-size check after every visual change.
Trademark and image licensing	Unpublishable project	Locally bundled assets, a generated credits screen and an explicit unofficial-use notice.
Stale documentation	Report describes code that no longer exists	Code listings, tables and build numbers are generated from the source tree and the build log.

3. Requirements

Eleven routes define the product. Everything else — services, tokens, scripts — exists to make those eleven screens fast, accurate and consistent.

3.1 Functional requirements

Each route is a complete reading task rather than a fragment of a dashboard. The table records the purpose of each screen and the component that implements it.

Table 5 — The eleven routed screens

ROUTE	SCREEN	WHAT IT MUST DO
`/`	Home	Pair the McLaren MP4/4 hero with the live 2026 top five, archive totals, the latest result and the next race.
`/standings`	Standings	Drivers' and constructors' tables for any season, a leader or champion card, points gap and a CSV export.
`/drivers`	Drivers	Searchable, sortable grid; profile dialog with career totals, titles, seasons and teams.
`/constructors`	Teams	Team grid with real logos or typographic crests, title years, line-ups and career totals.
`/circuits`	Circuits	Searchable library, world map, layout image and a detail panel with race history.
`/races`	Races	Season calendar with winners and podiums, plus a year switcher.
`/races/:id`	Race detail	Winner hero, podium, fastest lap, classification, qualifying tabs, previous/next round and season context.
`/live`	Live timing	OpenF1 meeting and session pickers, timing tower, weather, race control, lap telemetry and an optional token.
`/history`	History	Long-form article with infobox, five eras, four rivalries, champions table and records.
`/rivalry`	Rivalry	Two-driver picker, career totals, points chart, season table and a head-to-head score.
`/credits`	Credits	Every image, its source and licence, plus the trademark notice.

Two routes exist outside the table. `/timeline` is a redirect to `/history`, because the old timeline became the first section of the article. The wildcard route sends unknown URLs back to Home. Every page except Home is lazy-loaded, and every screen route sets its own document title.

3.2 Non-functional requirements

- **Speed.** The initial bundle must stay well inside the 1.5 MB warning budget, and archive views must not re-fetch CSV files per page.
- **Offline-friendly archive.** Once the bundle is cached, every 2000–2026 view works without a network call. Only the live page depends on OpenF1.
- **Responsive down to 390 px.** Tables may scroll horizontally, but navigation, headings and the background field must remain usable on a phone.
- **Accessibility.** Semantic headings, captioned tables, labelled controls, visible focus, a skip link, `aria-selected` tabs and a reduced-motion mode.
- **Accuracy.** No fabricated champions, no invented race winners, and no silent fallback that turns a missing value into a zero.

- **Maintainability.** One data-scope config, one token file, one shared component library, and generated documentation rather than hand-copied numbers.
- **Licensing.** Every downloaded image appears on the credits screen with its source; the app is labelled unofficial and non-commercial.

3.3 Deliberately out of scope

The project does not implement user accounts, write operations, a database in the runtime path, native mobile apps or internationalisation. Sprint results are not yet in the bundled archive, and true real-time polling requires an OpenF1 paid token. These are stated as limitations rather than hidden behind placeholder controls.

4. System architecture

The shipped application is a static Angular bundle with two data services: one for the bundled archive, one for OpenF1. There is no runtime server in the path.

4.1 Overview

The architecture is deliberately shallow. Pages own their presentation and local view state; two root-provided services own data access; a small config and model layer defines the vocabulary shared between them. Nothing else is allowed to fetch data, which keeps the CSV parsing and the OpenF1 rate limiting in exactly two places.

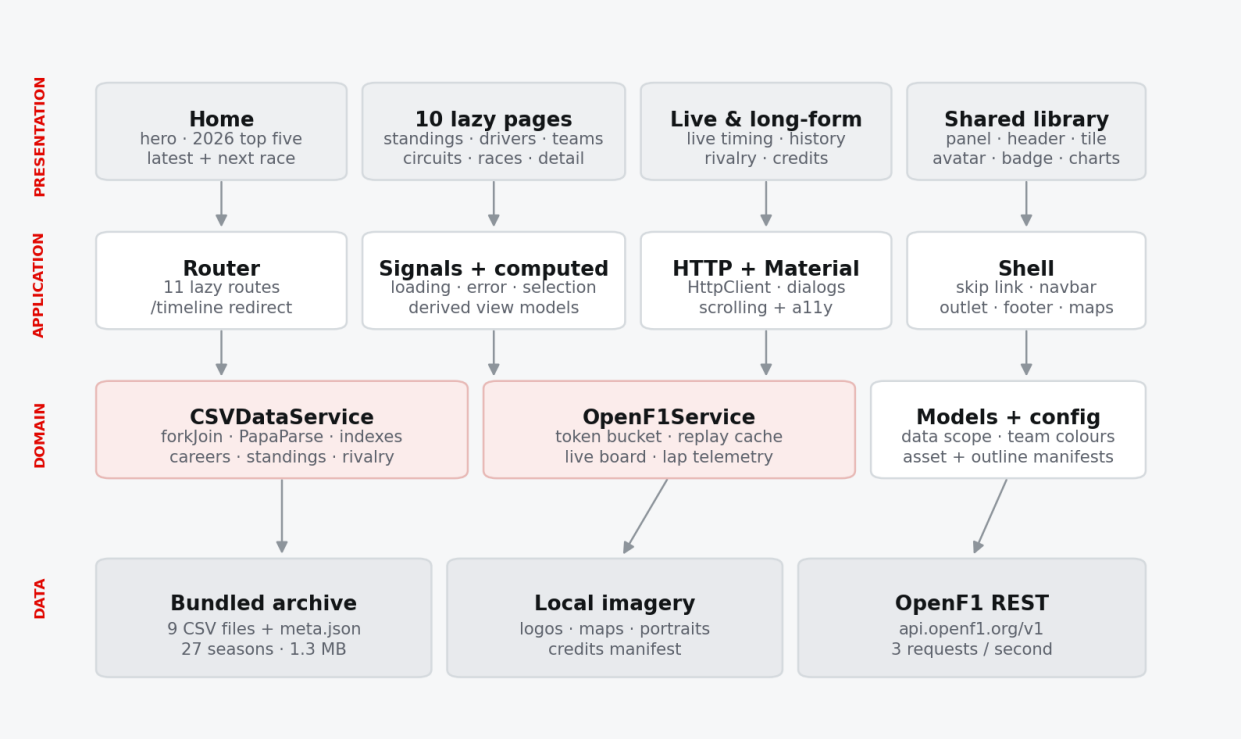


Figure 4 — Layered architecture. Pages depend on services; services depend on local assets or the OpenF1 API; the shell and shared library are used by every page.

The build-time tooling sits outside the runtime diagram: Node scripts produce the CSV archive, download imagery and trace the circuit outlines; Python scripts turn the same source tree into the README, the presentation and this report. The browser never runs any of it.

4.2 Application shell

`app.ts` is intentionally small. It renders a skip link, the fixed background layer, the navbar, a `<main>` element with the router outlet, and the footer. The skip link targets `#content`, and `<main>` carries `tabindex="-1"` so keyboard users can jump past the navigation. The background sits behind everything with `pointer-events: none`, so it can never intercept a click.

The shell imports `TrackBackgroundComponent` directly rather than through a route, because the field is part of the page frame and must survive navigation. The component draws its own fixed layer and

holds its positions in memory; changing route does not rebuild the field, so there is no flash between pages.

Global providers are registered in `app.config.ts`: the router with component input binding, view transitions and in-memory scroll restoration; `HttpClient`; browser animations; and the Material dialog module. The scroll restoration matters on long pages such as history and race detail, where returning from a driver dialog or a query-parameter change should not jump the reader back to the top.

4.3 Routing and lazy loading

Home is imported eagerly so the first paint does not wait for a second chunk. The other ten routes use `loadComponent`, and each route declares its own title. The route table is short enough to read as the product definition.

Listing 1 — The first routes: an eager Home, a lazy Standings page and per-route titles · typescript

```
export const routes: Routes = [
  {
    path: '',
    component: HomeComponent,
    title: 'Paddock — Formula 1 archive',
  },
  {
    path: 'standings',
    loadComponent: () => import('./pages/standings/standings.component').then((m) => m.StandingsComponent),
    title: 'Standings — Paddock',
  },
  {
```

The remaining routes follow the same pattern: `drivers`, `constructors`, `circuits`, `races`, `races/:id`, `live`, `history`, `rivalry` and `credits`. Two non-screen routes close the table: `/timeline` redirects to `/history`, and `**` returns the reader to Home. The parameterised race route is the only one that reads an `:id`; the archive's synthesised race identifiers make deep links stable across rebuilds.

4.4 Services and state

`CSVDataService` is provided at root and exposes `loading` and `error` signals alongside observable read methods. It parses the archive once, builds maps and derived arrays, and wraps the result in `shareReplay({ bufferSize: 1, refCount: false })`. A page that subscribes late still receives the same parsed object; a page that unsubscribes does not tear the cache down.

`OpenF1Service` is also root-provided, but its cache is a `Map` keyed by request path plus query. Historical responses are immutable, so they are kept for the lifetime of the tab. Live requests are cleared per session through `clearSessionCache` when the reader explicitly asks for fresh data.

Pages use a consistent pattern: injected services, `signal` for loaded rows, `computed` for derived view models, and `takeUntilDestroyed` for query-parameter subscriptions. The standings page is the clearest example — the year is a signal, the champion is a computed lookup, and the feature card falls back from champion to current leader without pretending the season is over.

4.5 Directory and shared library

The source tree separates four concerns. `core/` holds configuration, models, services and formatting utilities. `pages/` holds the routed screens. `features/` currently holds only the rivalry engine, which is large enough to deserve its own folder. `shared/components/` holds the visual primitives: panel, page header, stat tile, driver avatar, team badge, spark chart, telemetry chart, skeleton loader, empty state and the track background.

This split is not academic. Shared components are the only place where a surface, border radius or empty-state pattern is defined; pages compose them but do not re-implement them. That is what keeps eleven screens looking like one product, and it is why the design system could be re-skinned from the token file without editing every page.

4.6 Optional legacy backend

The repository still contains the original Express and MongoDB API under `backend/`, together with the untrimmed source dataset under `archive/`. It is not called by the Angular application. It is kept as a reference implementation and as a starting point for anyone who wants a server-side API; the README documents how to import the full dataset and run it on port 3000.

5. Data pipeline

Two pipelines meet in the browser: a build-time archive that is pruned and indexed once, and a runtime live layer that is rate-limited and cached.

5.1 Sources and scope

The bundled archive is built from two sources. **Ergast** supplies the historical dump through 2024; **Jolpica** supplies 2025 and 2026, including sprint-aware results and the new teams and circuits. The frontend never sees either API. It reads nine CSV files and one JSON manifest from `frontend/src/assets/archive/`.

Table 6 — The bundled archive

FILE	ROWS	ROLE
`drivers.csv`	130	Driver identity, code, nationality and date of birth
`constructors.csv`	40	Team identity and nationality
`circuits.csv`	39	Circuit identity, location and coordinates
`races.csv`	526	Season calendar across 27 seasons
`results.csv`	10,954	Race classification, grid, points, status and fastest lap
`qualifying.csv`	9,968	Q1, Q2 and Q3 times per race
`season_standings.csv`	27 seasons	Final or current drivers' standings
`season_constructor_standings.csv`	27 seasons	Final or current constructors' standings
`status.csv`	—	Result status labels such as Finished, Accident and Disqualified
`meta.json`	1	Generated timestamp, year range, complete years and row counts

The pruned set is about **1.3 MB**. That figure is the reason the architecture works: it is small enough to send once with the application, so every archive view can be answered locally. The original untrimmed dataset remains in `archive/` for the optional backend.

5.2 Build-time ingest

`prune-archive.mjs` reads the full dump, keeps rows from 2000 onward, rebuilds the season standings from the surviving results and writes the archive files plus `meta.json`. It never deletes from the source folder, so the operation is repeatable. `sync-current-season.mjs` then fetches 2025 and 2026 from Jolpica in pages of 100, appends races, results, qualifying and standings, and synthesises race identifiers as `year * 100 + round`. Because the identifiers are deterministic, deep links such as `/races/202618` survive a data refresh.

Two further scripts prepare the presentation layer. `fetch-assets.mjs` downloads team logos, circuit maps, driver portraits and archive photographs, skips files that already exist, backs off on HTTP 429, and writes `asset-manifest.ts` containing only the assets that actually landed on disk. `extract-`

`circuit-outlines.mjs` traces the circuit map images into the stroke-only SVG paths used by the background and by the circuit cards.

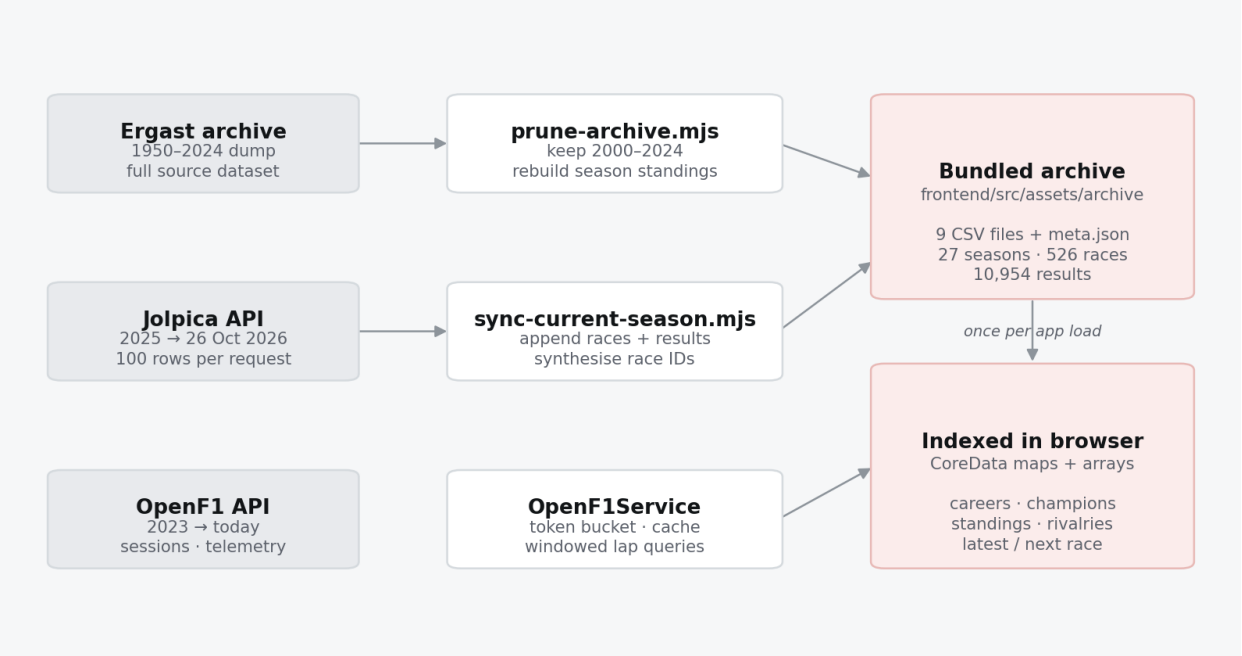


Figure 5 — Data flow. Build-time scripts produce the immutable archive; the browser indexes it once; OpenF1 is the only runtime data dependency.

5.3 Runtime indexing

The frontend loads all ten archive files in parallel with `forkJoin`, maps each row into a typed model, and indexes the result. The observable is created once and cached, so the work happens on the first subscription and never again.

Listing 2 — The archive is loaded in parallel, built once and replayed to every page · typescript

```
private core$: Observable<CoreData> = forkJoin({
  meta: this.loadJson<ArchiveMeta>('meta.json'),
  drivers: this.load('drivers.csv'),
  constructors: this.load('constructors.csv'),
  circuits: this.load('circuits.csv'),
  races: this.load('races.csv'),
  results: this.load('results.csv'),
  qualifying: this.load('qualifying.csv'),
  status: this.load('status.csv'),
  standings: this.load('season_standings.csv'),
  constructorStandings: this.load('season_constructor_standings.csv'),
}).pipe(
  map((raw) => this.build(raw as never)),
  tap((core) => {
    this.loading.set(false);
    this.error.set(null);
    if (typeof console !== 'undefined') {
      console.info(
        `[archive] ${core.driverCareers.length} drivers · ${core.constructorCareers.length} teams · ` +
          `${core.races.length} races · ${core.results.length} results`,
      );
    }
  }),
  catchError((err) => {
    this.loading.set(false);
    this.error.set(err?.message ?? 'Unable to load the archive');
    console.error('[archive] failed to load', err);
    return of(this.buildEmpty());
  }),
  shareReplay({ bufferSize: 1, refCount: false }),
);
```

```
// ---------------------------------------------------------------------
```

`build()` is the single place where raw strings become domain objects. It performs the numeric and text conversions, then constructs lookup maps and grouping maps that make the rest of the service cheap. There are no repeated linear scans in page code.

Listing 3 — Identity maps and grouped indexes built once per load · typescript

```
    const driverById = new Map(drivers.map((driver) => [driver.driverId, driver]));
    const constructorById = new Map(constructors.map((constructor) => [constructor.constructorId,
constructor]));
    const circuitById = new Map(circuits.map((circuit) => [circuit.circuitId, circuit]));
    const raceById = new Map(races.map((race) => [race.raceId, race]));
    const yearOfRace = new Map(races.map((race) => [race.raceId, race.year]));
    const statusById = new Map(raw.status.map((row) => [num(row['statusId']), text(row['status'])]));

    const resultsByRace = groupBy(results, (row) => row.raceId);
    const resultsByDriver = groupBy(results, (row) => row.driverId);
    const qualifyingByRace = groupBy(qualifying, (row) => row.raceId);
    const qualifyingByDriver = groupBy(qualifying, (row) => row.driverId);
    const standingsByDriver = groupBy(standings, (row) => row.driverId);
    const constructorStandingsByConstructor = groupBy(constructorStandings, (row) => row.constructorId);
```

5.4 Derived views

Careers, season summaries, race detail and rivalry data are derived from the indexes rather than stored. A driver's career is the union of their results, qualifying rows and season standings; a circuit's history is the set of races at that circuit; a rivalry is two careers reduced to a common set of seasons. Deriving instead of duplicating keeps the archive internally consistent.

The most important derived distinction is complete versus in-progress. A race counts as run only when it has results, and a season is complete only when `meta.json` lists it. The 2026 calendar is bundled in full, including rounds that have not happened, so “latest race” can never mean “last row in the file”.

Listing 4 — “Latest” and “next” are decided by results and dates, not by file order · typescript

```
    // A race counts as run when it has results. The 2026 calendar is bundled in
    // full, so "latest race" must never mean "last round in the file".
    const hasResults = (race: Race) => (resultsByRace.get(race.raceId)?.length ?? 0) > 0;
    const today = new Date().toISOString().slice(0, 10);
    const completed = races.filter(hasResults);

    core.latestRace = completed.length ? completed[completed.length - 1] : null;
    core.nextRace =
      races.find((race) => !hasResults(race) && race.date >= today) ??
      races.find((race) => !hasResults(race)) ??
      null;
```

The same rule reaches the standings page. `getChampions()` returns only seasons present in `completeYears`, while `getSeasonStandings()` returns the current table for an unfinished season. The UI can therefore show a **leader** card for 2026 and a **champion** card for 2025 without any special-case copy in the template.

5.5 OpenF1 integration

OpenF1 is used from 2023 onward. The service exposes typed methods for sessions, meetings, drivers, positions, intervals, laps, stints, pit stops, weather, race control, car data and location. Every method funnels through one private `get()` that applies the token bucket and the cache.

Listing 5 — A three-token bucket keeps the client under OpenF1's three requests per second · typescript

```
  private tokens = 3;
  private lastRefill = Date.now();

  // ---------------------------------------------------------------------------
  // Authentication / cache control
  // ---------------------------------------------------------------------------
  get hasToken(): boolean {
    return !!this.token;
  }

  setToken(token: string | null): void {
    const next = token?.trim() || null;
    this.token = next;
    try {
      if (next) localStorage.setItem(TOKEN_KEY, next);
      else localStorage.removeItem(TOKEN_KEY);
    } catch {
      // Storage may be unavailable; the token still works for this session.
    }
    this.cache.clear();
  }

  /** Drops cached responses for one session so polling sees fresh timing data. */
  clearSessionCache(sessionKey: number): void {
    for (const key of [...this.cache.keys()]) {
      if (key.includes(`:${sessionKey}`)) this.cache.delete(key);
    }
  }

  // ---------------------------------------------------------------------------
  // Discovery
  // ---------------------------------------------------------------------------
  getSession(sessionKey: number): Observable<OpenF1Session[]> {
    return this.get<OpenF1Session[]>(`/sessions?session_key=${sessionKey}`, `session:${sessionKey}`);
  }

  getSessions(year?: number): Observable<OpenF1Session[]> {
    return this.get<OpenF1Session[]>(year ? `/sessions?year=${year}` : '/sessions', `sessions:${year ??
'all'}`);
  }

  getMeetings(year: number): Observable<OpenF1Meeting[]> {
    return this.get<OpenF1Meeting[]>(`/meetings?year=${year}`, `meetings:${year}`);
  }

  getLatestSession(): Observable<OpenF1Session[]> {
    return this.get<OpenF1Session[]>('/sessions?session_key=latest', 'session:latest');
  }

  getDrivers(sessionKey: number): Observable<OpenF1Driver[]> {
    return this.get<OpenF1Driver[]>(`/drivers?session_key=${sessionKey}`, `drivers:${sessionKey}`).pipe(
      map((drivers) => drivers.slice().sort((a, b) => a.driver_number - b.driver_number)),
    );
  }

  // ---------------------------------------------------------------------------
  // Results / standings
  // ---------------------------------------------------------------------------
  getSessionResult(sessionKey: number): Observable<OpenF1SessionResult[]> {
    return this.get<OpenF1SessionResult[]>(`/session_result?session_key=${sessionKey}`,
`result:${sessionKey}`);
  }
```

```
  getDriversChampionship(sessionKey: number): Observable<OpenF1ChampionshipDriver[]> {
    return this.get<OpenF1ChampionshipDriver[]>(
      `/championship_drivers?session_key=${sessionKey}`,
      `champ-drivers:${sessionKey}`,
    );
  }

  getTeamsChampionship(sessionKey: number): Observable<OpenF1ChampionshipTeam[]> {
    return this.get<OpenF1ChampionshipTeam[]>(
      `/championship_teams?session_key=${sessionKey}`,
      `champ-teams:${sessionKey}`,
    );
  }

  getStartingGrid(sessionKey: number): Observable<{ position: number; driver_number: number; lap_duration:
number }[]> {
    return this.get(`/starting_grid?session_key=${sessionKey}`, `grid:${sessionKey}`);
  }

  // ---------------------------------------------------------------------------
  // Timing (all-driver endpoints — one request per table, never per driver)
  // ---------------------------------------------------------------------------
  getIntervals(sessionKey: number): Observable<OpenF1Interval[]> {
    return this.get<OpenF1Interval[]>(`/intervals?session_key=${sessionKey}`, `intervals:${sessionKey}`);
  }

  getPositions(sessionKey: number): Observable<OpenF1Position[]> {
    return this.get<OpenF1Position[]>(`/position?session_key=${sessionKey}`, `positions:${sessionKey}`);
  }

  getLaps(sessionKey: number, driverNumber?: number): Observable<OpenF1Lap[]> {
    const query = driverNumber
      ? `/laps?session_key=${sessionKey}&driver_number=${driverNumber}`
      : `/laps?session_key=${sessionKey}`;
    return this.get<OpenF1Lap[]>(query, `laps:${sessionKey}:${driverNumber ?? 'all'}`);
  }

  getStints(sessionKey: number): Observable<OpenF1Stint[]> {
    return this.get<OpenF1Stint[]>(`/stints?session_key=${sessionKey}`, `stints:${sessionKey}`);
  }

  getPitStops(sessionKey: number): Observable<OpenF1Pit[]> {
    return this.get<OpenF1Pit[]>(`/pit?session_key=${sessionKey}`, `pit:${sessionKey}`);
  }

  getRaceControl(sessionKey: number): Observable<OpenF1RaceControl[]> {
    return this.get<OpenF1RaceControl[]>(`/race_control?session_key=${sessionKey}`, `race-
control:${sessionKey}`);
  }

  getWeather(sessionKey: number): Observable<OpenF1Weather[]> {
    return this.get<OpenF1Weather[]>(`/weather?session_key=${sessionKey}`, `weather:${sessionKey}`);
  }

  // ---------------------------------------------------------------------------
  // Telemetry (windowed — a full session is far too large)
  // ---------------------------------------------------------------------------
  getCarData(sessionKey: number, driverNumber: number, from: string, to: string): Observable<OpenF1CarData[]>
{
    const query =
      `/car_data?session_key=${sessionKey}&driver_number=${driverNumber}` +
      `&date>=${encodeURIComponent(from)}&date<=${encodeURIComponent(to)}`;
    return this.get<OpenF1CarData[]>(query, `car:${sessionKey}:${driverNumber}:${from}:${to}`);
  }

  getLocation(sessionKey: number, driverNumber: number, from: string, to: string):
Observable<OpenF1Location[]> {
    const query =
      `/location?session_key=${sessionKey}&driver_number=${driverNumber}` +
      `&date>=${encodeURIComponent(from)}&date<=${encodeURIComponent(to)}`;
    return this.get<OpenF1Location[]>(query, `loc:${sessionKey}:${driverNumber}:${from}:${to}`);
  }

  getLapTelemetry(
```

```
  sessionKey: number,
  driverNumber: number,
  from: string,
  to: string,
): Observable<{ car: OpenF1CarData[]; location: OpenF1Location[] }> {
  return forkJoin({
    car: this.getCarData(sessionKey, driverNumber, from, to),
    location: this.getLocation(sessionKey, driverNumber, from, to),
  });
}

// ---------------------------------------------------------------------------
// Composed view for the live board
// ---------------------------------------------------------------------------
getLiveBoard(sessionKey: number): Observable<{
  drivers: OpenF1Driver[];
  rows: LiveDriverRow[];
  laps: OpenF1Lap[];
  raceControl: OpenF1RaceControl[];
  weather: OpenF1Weather | null;
  result: OpenF1SessionResult[];
}> {
  return forkJoin({
    drivers: this.getDrivers(sessionKey),
    intervals: this.getIntervals(sessionKey),
    positions: this.getPositions(sessionKey),
    laps: this.getLaps(sessionKey),
    stints: this.getStints(sessionKey),
    pit: this.getPitStops(sessionKey),
    raceControl: this.getRaceControl(sessionKey),
    weather: this.getWeather(sessionKey),
    result: this.getSessionResult(sessionKey),
  }).pipe(
    map(({ drivers, intervals, positions, laps, stints, pit, raceControl, weather, result }) => {
      const rows = buildLiveRows(drivers, intervals, positions, laps, stints, pit, result);
      return {
        drivers,
        rows,
        laps,
        raceControl: raceControl.slice(-40).reverse(),
        weather: weather[weather.length - 1] ?? null,
        result,
      };
    }),
  );
}

// ---------------------------------------------------------------------------
// Plumbing
// ---------------------------------------------------------------------------
private get<T>(path: string, key: string): Observable<T> {
  const cached = this.cache.get(key);
  if (cached) return cached as Observable<T>;

  const request = new Observable<T>((subscriber) => {
    let cancelled = false;

    this.acquire().then(() => {
      if (cancelled) return;

      const headers = this.token
        ? new HttpHeaders({ Authorization: `Bearer ${this.token}` })
        : undefined;

      firstValueFrom(this.http.get<T>(`${BASE}${path}`, { headers }))
        .then((data) => {
          if (cancelled) return;
          subscriber.next(data);
          subscriber.complete();
        })
        .catch((err) => {
          if (cancelled) return;
          subscriber.error(err);
        });
    });

    return () => {
```

```
      cancelled = true;
    };
  }).pipe(shareReplay({ bufferSize: 1, refCount: false }));

  this.cache.set(key, request as Observable<unknown>);
  return request;
}

private async acquire(): Promise<void> {
  for (;;) {
    const now = Date.now();
    const elapsed = now - this.lastRefill;
    if (elapsed > 0) {
      this.tokens = Math.min(3, this.tokens + elapsed / 340);
      this.lastRefill = now;
    }
    if (this.tokens >= 1) {
      this.tokens -= 1;
      return;
    }
```

The bucket starts with three tokens, refills at one token every 340 ms (about 2.9 requests per second) and waits in 90 ms steps when empty. The cache is checked before the bucket, so a repeated historical request costs nothing. Requests are wrapped in `shareReplay({ bufferSize: 1, refCount: false })` and cancelled through the observable teardown when a component unsubscribes.

Listing 6 — Cache first, then throttle, then one HTTP request shared by all subscribers · typescript

```
private get<T>(path: string, key: string): Observable<T> {
  const cached = this.cache.get(key);
  if (cached) return cached as Observable<T>;

  const request = new Observable<T>((subscriber) => {
    let cancelled = false;

    this.acquire().then(() => {
      if (cancelled) return;

      const headers = this.token
        ? new HttpHeaders({ Authorization: `Bearer ${this.token}` })
        : undefined;

      firstValueFrom(this.http.get<T>(`${BASE}${path}`, { headers }))
        .then((data) => {
          if (cancelled) return;
          subscriber.next(data);
          subscriber.complete();
        })
        .catch((err) => {
          if (cancelled) return;
          subscriber.error(err);
        });
    });

    return () => {
      cancelled = true;
    };
  }).pipe(shareReplay({ bufferSize: 1, refCount: false }));

  this.cache.set(key, request as Observable<unknown>);
```

The live board composes nine endpoints in one `forkJoin`: drivers, intervals, positions, laps, stints, pit stops, race control, weather and the final session result. `buildLiveRows()` merges the latest row per driver into a single timing row, race control is trimmed to the last forty messages and reversed, and

the latest weather sample is kept. Telemetry is never downloaded for a whole session: the page requests one driver and one lap inside a date window.

5.6 Changing the scope

The archive window is not scattered through the code. Three constants define it, and the year list is computed from them. Changing the start year is therefore a data refresh plus a one-line edit, not a search-and-replace across pages.

Listing 7 — Data scope is centralised; the live bundle value still comes from meta.json · typescript

```
export const DATA_START_YEAR = 2000;

/** Last season present in the bundle. The live value comes from `meta.endYear`. */
export const ARCHIVE_END_YEAR = 2026;

/** Date the archive was last reconciled with the live season. */
export const DATA_THROUGH = '2026-10-26';

/** OpenF1 (and therefore live timing / telemetry) starts here. */
export const OPENF1_START_YEAR = 2023;

export const CURRENT_YEAR = new Date().getFullYear();

/** Every season the UI offers, newest first. */
export const AVAILABLE_YEARS: number[] = Array.from(
  { length: ARCHIVE_END_YEAR - DATA_START_YEAR + 1 },
  (_, i) => ARCHIVE_END_YEAR - i,
);
```

6. Frontend implementation

Eleven screens share one composition pattern: a page header, a small number of panels, one clear primary table or chart, and a single primary action.

6.1 Page composition

Every page follows the same rhythm. `PageHeaderComponent` carries the eyebrow, title, description and optional actions. `PanelComponent` groups related content on a matte silver surface with a hairline border. `StatTileComponent` presents a single number with a label and optional delta. Tables use the global table styles; charts use the shared SVG components. Loading states use skeleton blocks, and empty states use the shared empty-state panel rather than a bare sentence.

This is why the pages can be large without becoming inconsistent. The live page is the biggest file in the project at 1,343 lines, the history article is 972 and race detail is 825, but most of those lines are markup and scoped styles for a specific reading task. The shared decisions — surface, border, focus ring, table rhythm, typography — live outside them.

Table 7 — The largest source files and what they own

FILE	LINES	RESPONSIBILITY
`pages/live/live.component.ts`	1,343	Session picker, timing tower, weather, race control, telemetry, token panel
`pages/history/history.component.ts`	972	Long-form article, infobox, eras, rivalries, champions, records
`pages/race-detail/race-detail.component.ts`	825	Winner hero, podium, classification, qualifying tabs, round navigation
`pages/home/home.component.ts`	782	Hero, top five, archive totals, latest and next race
`pages/standings/standings.component.ts`	762	Drivers' and constructors' tables, leader card, year switcher, export
`core/services/csv-data.service.ts`	746	Archive load, indexes, careers, standings, rivalry, summaries
`features/rivalry/rivalry.component.ts`	707	Two-driver picker, career comparison, chart and season table
`pages/races/races.component.ts`	696	Calendar, winners, podiums, year switcher
`pages/constructors/constructors.component.ts`	664	Team grid, crests, title years, line-ups
`pages/circuits/circuits.component.ts`	648	Map, searchable library, circuit detail
`pages/drivers/drivers.component.ts`	533	Searchable, sortable driver grid
`components/navbar/navbar.component.ts`	415	Desktop navigation, mobile sheet, active rule

6.2 Home

Home is the only eager route, and it carries the most editorial weight. The hero pairs a cut-out McLaren MP4/4 with the 2026 championship top five, so the archive's history and its present are

visible in the same frame. Below that, three blocks answer the questions a reader arrives with: how large the archive is, what happened in the latest race, and where the next round takes place.

The hero does not call the 2026 leader a champion. It uses the season summary to show rounds completed, the current leader, the gap to second and the next race. The 2025 champion feature is separate and explicitly labelled, because 2025 is a complete season. The archive totals are read from `meta.json` rather than hard-coded, so a data refresh updates the copy automatically.

6.3 Standings

The standings page is the best illustration of the signal-based page pattern. The selected year is a signal initialised from the query string; champions and drivers are loaded from the service; the feature card, runner-up, margin and title count are computed. Changing the year writes `?year=` with `replaceUrl`, so the back button does not walk through every season the reader browsed.

Listing 8 — Derived standings state: a champion only when the season is complete · typescript

```
  readonly champion = computed(() => this.champions().find((row) => row.year === this.year()) ?? null);
  readonly seasonComplete = computed(() => this.champion() !== null);
  /** Champion if the season is decided, otherwise the current leader. Never a fabricated title. */
  readonly feature = computed(() => this.champion() ?? this.drivers()[0] ?? null);
  readonly roundsLeft = computed(() => {
    const summary = this.summary();
    return summary ? Math.max(0, summary.roundsTotal - summary.roundsCompleted) : 0;
  });
  readonly runnerUp = computed<StandingsRow | null>(() => this.drivers()[1] ?? null);
  readonly teamChampion = computed(() => this.teams()[0] ?? null);
  readonly margin = computed(() => {
    const champion = this.feature();
    const runnerUp = this.runnerUp();
    if (!champion || !runnerUp) return 0;
    return Math.max(0, Math.round((champion.points - runnerUp.points) * 10) / 10);
  });
  readonly titles = computed(() => {
    const driverId = this.champion()?.driver?.driverId;
    if (!driverId) return 0;
    return this.champions().filter((row) => row.driver?.driverId === driverId).length;
  });
```

The page shows both championships, a leader or champion card, the points margin and a season switcher. It also builds a CSV export in the browser from the loaded rows, which is useful for coursework and avoids a server-side report endpoint.

6.4 Drivers and teams

The drivers page is a searchable, sortable grid. Each card opens a profile dialog with career totals, titles, the seasons contested and the teams driven for. The dialog is a Material component but uses the same silver surfaces and table styles as the rest of the app, so it does not feel like a foreign layer.

The teams page mirrors the structure with a logo or a typographic crest, title years, driver line-ups and career totals. The crest is not a placeholder image: it is a deliberate fallback for the five teams without a licensable mark — MF1, Spyker MF1, Jaguar, Lotus Racing and Marussia. The 2026 grid includes the new Audi and Cadillac entries, both of which carry real marks.

6.5 Circuits and races

The circuits page combines three representations of the same object: a world map for geographic context, a searchable library of circuit cards, and a detail panel with the layout, location and race history. The outline used in the background is the same traced path family used on the cards, which ties the decoration to the content.

The races page is a season calendar. Each round shows its winner and podium when results exist, and a clear “upcoming” treatment when they do not. The year switcher uses the same query-parameter pattern as standings. Cancelled 2026 rounds keep their calendar entries but never show a winner, because a race without results cannot produce a classification.

6.6 Race detail

Race detail is the deepest archive screen. It opens with the winner, then the podium, fastest lap and the full classification. Qualifying is presented in tabs for Q1, Q2 and Q3. Previous and next round links make the calendar navigable without returning to the races page, and a context strip places the race in its season and at its circuit.

The screen resolves its race from the `:id` parameter and combines the race row, its results, its qualifying rows, the circuit and the season summary. Missing values are rendered as dashes rather than zeros, which matters for retirements, disqualifications and races where a driver did not set a time.

6.7 Live timing

The live page is a replay tool first and a live poller second. The reader chooses a season, meeting and session; the selection is mirrored into the URL with `replaceUrl`, so a session can be linked and the back button behaves. The timing tower merges positions, intervals, laps, stints, pit stops and the final classification into one row per driver.

Listing 9 — One forkJoin composes the live board from nine OpenF1 endpoints · typescript

```
  getLiveBoard(sessionKey: number): Observable<{
    drivers: OpenF1Driver[];
    rows: LiveDriverRow[];
    laps: OpenF1Lap[];
    raceControl: OpenF1RaceControl[];
    weather: OpenF1Weather | null;
    result: OpenF1SessionResult[];
  }> {
    return forkJoin({
      drivers: this.getDrivers(sessionKey),
      intervals: this.getIntervals(sessionKey),
      positions: this.getPositions(sessionKey),
      laps: this.getLaps(sessionKey),
      stints: this.getStints(sessionKey),
      pit: this.getPitStops(sessionKey),
      raceControl: this.getRaceControl(sessionKey),
      weather: this.getWeather(sessionKey),
      result: this.getSessionResult(sessionKey),
    }).pipe(
      map(({ drivers, intervals, positions, laps, stints, pit, raceControl, weather, result }) => {
        const rows = buildLiveRows(drivers, intervals, positions, laps, stints, pit, result);
        return {
          drivers,
          rows,
          laps,
```

```
          raceControl: raceControl.slice(-40).reverse(),
          weather: weather[weather.length - 1] ?? null,
          result,
        };
      }),
    );
  }
```

Weather shows the latest sample, race control is trimmed to the last forty messages and reversed so the newest is first, and missing fields stay as dashes. Telemetry is a separate, explicit action: the reader picks a driver and a lap, and the service requests only that window of car data. The resulting trace shows speed, gear, DRS and summary statistics.

Polling is off by default. A bearer token can be pasted into the token panel; it is stored under `paddock.openf1.token`, sent only to `api.openf1.org`, and never committed. The polling preference is stored separately under `paddock.openf1.polling`. Without a token the page remains a complete historical replay, which is the state most readers will use.

6.8 History

The history page is a deliberately long-form article rather than a dashboard. It opens with a lead and an infobox, then moves through five eras, four defining rivalries, a champions table and a records section. Source Serif 4 is used for the article body while Titillium Web carries labels and tables, which gives the page the feel of a printed reference without breaking the application's typography.

The timeline that once lived at `/timeline` is now the first section of this article; the old route redirects here. That decision removed a thin screen and gave the history page a stronger opening.

6.9 Rivalry

The rivalry engine accepts two driver identifiers through `?d1=` and `?d2=`. It compares career totals, titles, wins, podiums, poles and fastest laps, draws a points chart across the seasons both drivers contested, and lists a season-by-season table. The head-to-head score is computed from race results where both drivers were classified, so it does not count a race one of them missed.

There is no hard-coded default pairing: the page opens with two empty selectors and a random-pair action, and a shared URL such as `?d1=1&d2=830` restores a comparison. Query values are validated against the loaded driver list, so a stale link cannot produce a broken panel.

6.10 Credits

Credits is generated from the same asset manifest that drives the images. It lists every team logo, circuit map, portrait and archive photograph with its source and licence, then states the trademark position: Formula 1, the F1 logo, team marks and circuit names belong to Formula One Licensing B.V., the FIA and the respective teams; they appear for identification and commentary only. The page is the reason the project can use real imagery responsibly.

6.11 Shared components and conventions

The shared library is small on purpose. `PanelComponent` owns the surface, border and padding. `PageHeaderComponent` owns the eyebrow, title and description rhythm. `StatTileComponent` owns the number/label pair. `DriverAvatar` reads the manifest and falls back to a monogram. `TeamBadge` does the same with a typographic crest. `SparkChartComponent` and `TelemetryChartComponent` draw SVG directly, with no charting dependency. `SkeletonLoaderComponent` and `EmptyStateComponent` make loading and absence look intentional rather than broken.

Two conventions are enforced across pages. First, components never hard-code a surface colour; they use the tokens. Second, interactive controls keep a visible focus ring and enough padding to be operated comfortably, including the icon-only buttons in the navbar and the session picker. These are small decisions, but they are what make eleven independently built screens feel like one application.

7. Design system

The visual system is a reaction against the default racing dashboard: no dark glass, no neon, no gradient wash. Silver, ink, hairline rules and one red accent.

7.1 Tokens

All colour, radius, elevation and motion values are declared once in `styles/_tokens.scss`. Components consume the custom properties; they do not invent values. The palette is intentionally short: a red brand scale, a six-step silver scale, a five-step ink scale, and semantic tokens for surfaces, text, dividers and focus.

Listing 10 — The core palette: one red, a silver scale and a single light colour scheme · typescript

```
:root {
  color-scheme: light;

  // Brand ---------------------------------------------------------------------
  --f1-red: #e10600;
  --f1-red-deep: #b30500;
  --f1-red-soft: #fbe8e7;

  // Silver scale --------------------------------------------------------------
  --silver-0: #f6f7f8; // page
  --silver-1: #eef0f2; // recessed wells
  --silver-2: #e5e8eb; // secondary surface
  --silver-3: #d5dade; // borders
  --silver-4: #bcc2c8;
  --silver-5: #8d949b;
```

The file opens with `color-scheme: light`, and there is no dark-mode branch anywhere in the project. That is a statement of intent: the archive is a printed object, and a second theme would double the surface decisions without improving the reading experience.

Team colours are stored separately in `core/config/data.config.ts` and applied only to badges, data bars and two-pixel rules. They never become page backgrounds, because forty team palettes would destroy the consistency the silver system provides.

7.2 Typography

Titillium Web carries the interface: navigation, labels, headings, tables and numbers. Source Serif 4 carries long-form article copy on the history page. Numerals in timing tables and standings are tabular, so columns of times and points line up without manual padding. Code and metadata use a monospace stack, which is why eyebrows and document labels look like paddock signage.

The type scale is fluid at the top end: page titles use `clamp()` so they shrink on a phone without a separate mobile stylesheet. Body copy stays at a readable measure; long paragraphs are not allowed to run the full width of a 1600 px viewport.

7.3 Surfaces and elevation

Every page is built from the same `.panel` primitive: flat matte silver, a hairline border, a two- to four-pixel radius and one soft shadow reserved for overlays. Brushed metal is used only on the home hero plate and the navbar, where it signals “object” rather than “card”. Tables sit directly on the panel surface with alternating row tint, not in nested cards.

This restraint is what allows the background field to be bold. Because the content surfaces are quiet and opaque, thirty outlines at 20% ink can sit behind them without making the text unreadable.

7.4 Red as signal

Red appears in exactly four places: the F1 mark, the active navigation rule, a winning row (P1), and one primary action per view. It is never a background, never a gradient and never decorative. The result is that red still means something when it appears — a reader can scan a page and know where the action is.

7.5 The background algorithm

The background is generated from thirty circuit identifiers, one for every generated outline except Buddh. A deterministic hash places each track in a loose slot, so the field looks organic but is identical on every load. Alternating rows are offset by half a cell, which prevents the columns from lining up mechanically.

Listing 11 — Slot construction: a staggered field with stable, hash-driven jitter · typescript

```
function buildSlot(
  index: number,
  columns: number,
  rows: number,
  size: [number, number],
  offset: [number, number],
): Slot {
  const column = index % columns;
  const row = Math.floor(index / columns);
  const stagger = row % 2 === 0 ? 0 : 0.5;
  const sizePx = size[0] + hash(index + 17) * (size[1] - size[0]);


  return {
    left: ((column + stagger + 0.5) / columns) * 100 - offset[0] + (hash(index + 3) - 0.5) * 3,
    top: ((row + 0.5) / rows) * 100 - offset[1] + (hash(index + 9) - 0.5) * 2.5,
    size: Math.round(sizePx),
  };
}


/**
```

The placement pass assigns each track a size between 125 and 195 px, a rotation between −35° and +35°, a drift of 5–9 px, a rock of 2–4° and a loop between 40 and 80 seconds. Every item shares the same 20% ink; variation comes from position, scale and rotation, not from opacity. The whole field is controlled by one `--ink` custom property, so the density can be tuned in one line.

Listing 12 — One ink value, one colour and one long, shallow float animation · typescript

```
      .tracks {
        position: absolute;
```

```
    inset: 0;
    /** Single ink control for the whole field. */
    --ink: 0.2;
    color: #0b0d0f;
  }

  .tracks__item {
    position: absolute;
    left: var(--left);
    top: var(--top);
    width: var(--size);
    height: var(--size);
    opacity: var(--ink);
    rotate: var(--rot);
    animation: trackFloat var(--dur) cubic-bezier(0.45, 0, 0.55, 1) var(--delay) infinite;
  }

  .tracks__item path {
    fill: none;
    stroke: currentColor;
    stroke-width: 1.05;
    stroke-linecap: round;
    stroke-linejoin: round;
    vector-effect: non-scaling-stroke;
  }

  @keyframes trackFloat {
    0%,
    100% {
      translate: 0 0;
      rotate: var(--rot);
    }

    50% {
      translate: var(--dx) var(--dy);
      rotate: calc(var(--rot) + var(--spin));
    }
  }
```

The animation moves each track by only a few pixels and rocks it a couple of degrees. That is deliberate: a field of thirty shapes with large motion would pull attention away from the content. The loops are long and the delays are negative, so the field is already mid-motion on first paint rather than starting in unison.

Responsiveness rebuilds the field instead of cropping it. At 1400 px the component switches to a five-column, twenty-item layout; at 760 px it switches to two columns and ten items. The alternative — hiding items at the edges — would leave the phone view with a bare strip, which is exactly the failure the third design was meant to fix.

Table 8 — Measured background behaviour at the three supported widths

VIEWPORT	VISIBLE MAPS	INK COVERAGE	INK CENTROID	DARKEST STROKE
1600 × 1100	30	2.51%	49% / 51%	199
1280 × 800	20	2.65%	50% / 53%	199
390 × 844	10	3.59%	55% / 47%	200

The measurements are taken from the rendered screenshots, not from the source. Coverage is the share of sampled pixels darkened by the field; the centroid is the ink-weighted centre of the frame. A centroid near 50/50 and a quadrant ratio near 1:1 are the pass condition for “spread across the viewport”.

7.6 Motion

Motion is short and eased. Page transitions use the router's view transitions; entrances are brief opacity and translate changes; the only looping motions in the content layer are the live indicator and the loading shimmer. The background loop is the exception, and it is explicitly disabled under `prefers-reduced-motion`.

The background and content motion are expressed in CSS rather than JavaScript timers, so the browser can suspend them when the tab is hidden.

7.7 Responsive behaviour

The layout is fluid down to 390 px. Navigation collapses into a Material sheet; grids reduce their column count; wide tables scroll horizontally inside a labelled region rather than forcing the page to overflow; hero plates stack; and the background field rebuilds. The phone layout is not a shrunken desktop — the reading order and the number of visible maps both change.

Touch targets are kept at a usable size even when the visual control is small, and hover-only affordances are duplicated with focus states. The result is a page that can be operated with a keyboard, a mouse or a thumb without a separate mobile build.

8. Imagery and licensing

Real logos, real circuit maps and real photographs make an archive feel credible. They also create the project's clearest legal and engineering constraints.

8.1 The asset manifest

Images are not referenced directly from templates. `scripts/fetch-assets.mjs` downloads them, records what actually exists in `core/config/asset-manifest.ts`, and the components consult that manifest before requesting a file. A missing asset therefore produces a designed fallback, never a broken image icon.

Table 9 — Bundled imagery and its fallbacks

SET	COUNT	SOURCE	FALLBACK
Team logos	35 of 40	Formula 1 media CDN	Typographic crest
Circuit maps	39 of 39	Formula 1 media CDN	Country flag
Driver portraits	28	OpenF1	Monogram
Archive photographs	9	Wikimedia Commons (CC / public domain)	Caption only
F1 mark	1	Formula 1	—

The five teams without a licensable mark are MF1, Spyker MF1, Jaguar, Lotus Racing and Marussia. Their typographic crests are not placeholders to be replaced later; they are the intended treatment for a historical grid that no longer has a consistent set of vector marks.

8.2 Why the archive is local

Hot-linking images from a media CDN would make the archive slower, fragile and harder to audit. Downloading once and shipping the files means the credits page can list exactly what is present, the build has no runtime dependency on a third-party image host, and the screenshots used in this report are reproducible.

The downloader is deliberately polite: it skips files that already exist, backs off on HTTP 429 and writes the manifest only after the downloads settle. The manifest is the single source of truth for both the components and the credits screen, so an image cannot appear in the interface without also appearing in the credits list.

8.3 Credits and trademark position

The credits route lists every image set with its source and licence, then states the position in plain language: Formula 1, F1, the F1 logo, team marks and circuit names are trademarks of Formula One Licensing B.V., the FIA and the respective teams. They appear for identification and commentary only. The project is unofficial, non-commercial coursework.

This is not legal advice, and the report does not claim that the use is risk-free. It is an honest attempt to attribute what can be attributed, to use openly licensed material where possible, and to make the project's status clear to anyone who opens it.

9. Verification and testing

Verification was build-based and route-based: compile the production bundle, load every route in a real browser, check the content and the background, then inspect the generated documents.

9.1 Build health

The production build is the first gate. It must complete without errors, stay inside the configured budgets and produce a bundle whose initial transfer size is acceptable on a slow connection. The figures below are from the build used to capture the screenshots in this report.

Table 10 — Production build and route checks

METRIC	RESULT	NOTES
Production build	Pass	Angular CLI 21 / esbuild, no errors or budget warnings
Initial bundle, raw	580.38 kB	Framework, Home and shared library
Initial bundle, transferred	154.23 kB	Compressed estimate from the build log
Largest lazy chunk	Under 45 kB	Live timing is the largest route chunk
Routes rendered	11 of 11	Home plus ten lazy routes
Background items per route	30	All unique circuit identifiers

9.2 Route smoke tests

Each route was loaded from the production preview in headless Edge. The check confirmed that the page rendered its expected content, that the background component mounted, and that no route fell through to the wildcard redirect. The terms below are the content probes used for each screen.

Table 11 — Headless route smoke tests against the production build

ROUTE	CONTENT PROBE	BACKGROUND	RESULT
`/`	MP4/4 · Antonelli · 2026 · Drivers	30 items	Pass
`/standings?year=2026`	Antonelli · Championship leader · Mercedes	30 items	Pass
`/drivers`	Antonelli · Verstappen	30 items	Pass
`/constructors`	Mercedes · Audi · Cadillac	30 items	Pass
`/circuits`	Monza · Madring	30 items	Pass
`/races?year=2026`	Bahrain · Mexico City	30 items	Pass
`/races/202618`	Classification · Podium	30 items	Pass
`/live?year=2026`	Live timing · Telemetry	30 items	Pass
`/history`	Senna · Lauda · MP4/4	30 items	Pass
`/rivalry`	Choose two drivers · Rivalry	30 items	Pass
`/credits`	Trademark notice · licence	30 items	Pass

The background check also counts unique identifiers in the component: thirty identifiers, thirty unique values. That is what prevents a repeated circuit from looking like a richer field than it is.

Visibility was checked at three widths, confirming 30 visible items at 1600 px, 20 at 1280 px and 10 at 390 px.

9.3 Visual measurement

The background was verified from the captured PNGs rather than from the CSS. A small image analysis script computed ink coverage, the ink-weighted centroid, the quadrant distribution and the darkest stroke. Those measurements are the evidence behind the claims in section 7.5 and the reason the corner-collage design was rejected in favour of the field.

This is a pragmatic compromise. The build agent cannot judge composition by eye, so density and balance are expressed as numbers that can fail. The final aesthetic judgement still belongs to a human reader, which is why the screenshots are included at full width in section 10.

9.4 Screenshot capture

The eleven screenshots were captured with headless Microsoft Edge at **1600 × 1100**, using `--virtual-time-budget=30000` and `--run-all-compositor-stages-before-draw` so that fonts, images and the background field had settled before the frame was written. Each route was captured at the same viewport, which makes the set comparable and keeps the report's figure widths consistent.

The live screenshot uses the replay state rather than a paid live session, so the capture is reproducible and does not depend on a race weekend. The home screenshot is used on the project details page; the full set appears in section 10.

9.5 Document verification

The README, presentation and this report are generated artefacts, so each one is checked after it is built. The README and deck are exported to PDF through Microsoft Office, the page and slide counts are recorded, the text is extracted and searched for key claims, and the deck's shape bounds are checked to catch content that has slid off a slide.

Table 12 — Generated-document checks

ARTEFACT	CHECK	RESULT
README.docx / README.pdf	12 pages, text and figures present	Pass
f1-presentation.pptx / .pdf	15 slides, all shapes inside slide bounds	Pass
Micro_Project_Report.docx / .pdf	All chapters, 12 listings, 11 screenshots, TOC refreshed	Pass
Screenshots	11 files at 1600 × 1100	Pass

The verification is not a committed unit-test suite. There are no Angular TestBed specs, no end-to-end runner and no automated accessibility audit in the repository. The checks above are scripted observations of the production build. That is a real limitation, and it is stated here rather than implied by a green table.

10. Screenshots

Eleven screens, captured at 1600 × 1100 from the production build. Each caption gives the route, the reading task and the detail worth noticing.

Table 13 — Screenshot index

FIGURE	FILE	ROUTE
Figure 6	home.png	/
Figure 7	standings.png	/standings?year=2026
Figure 8	drivers.png	/drivers
Figure 9	teams.png	/constructors
Figure 10	circuits.png	/circuits
Figure 11	races.png	/races?year=2026
Figure 12	race-detail.png	/races/202618
Figure 13	live.png	/live?year=2026
Figure 14	history.png	/history
Figure 15	rivalry.png	/rivalry
Figure 16	credits.png	/credits

10.1 Archive screens

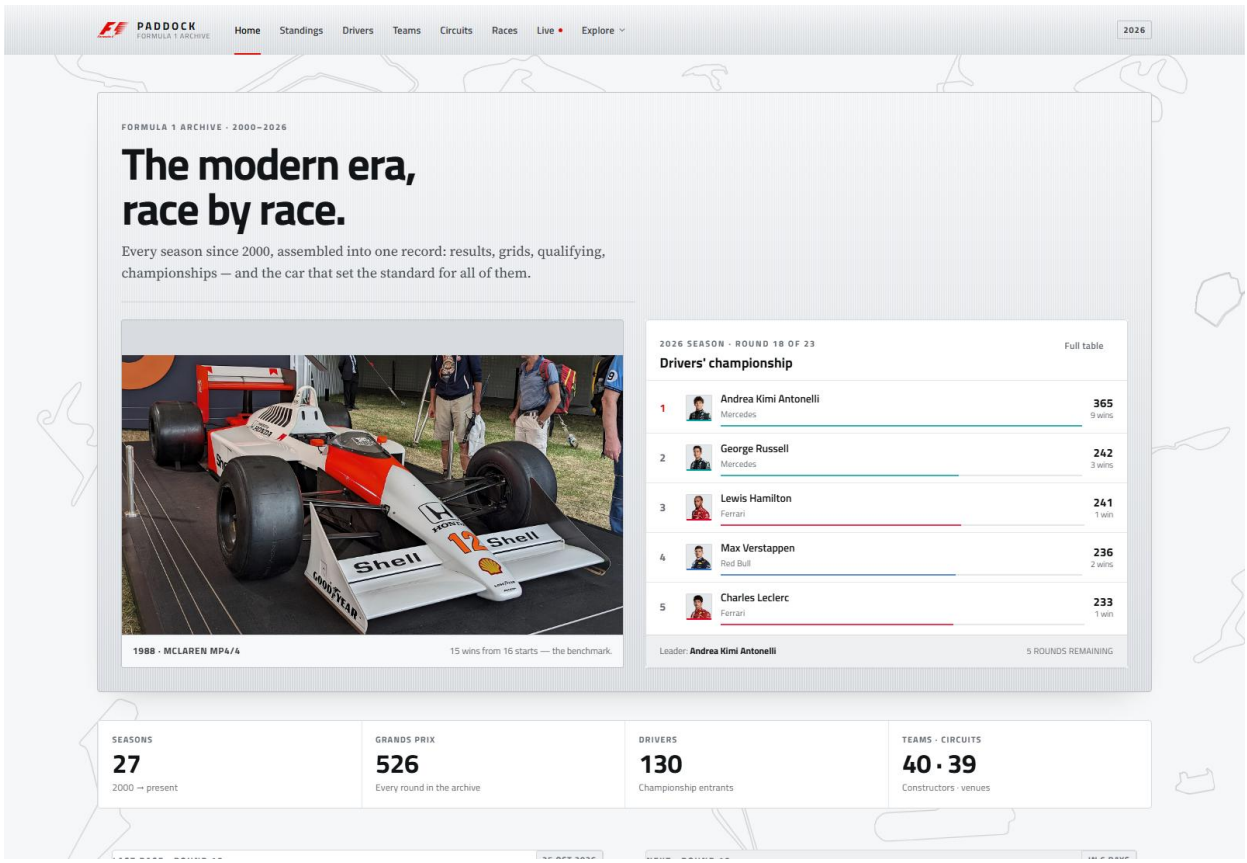


Figure 6 — Home (/). The MP4/4 hero, the 2026 top five, archive totals and the latest and next race. This is the only eager route.

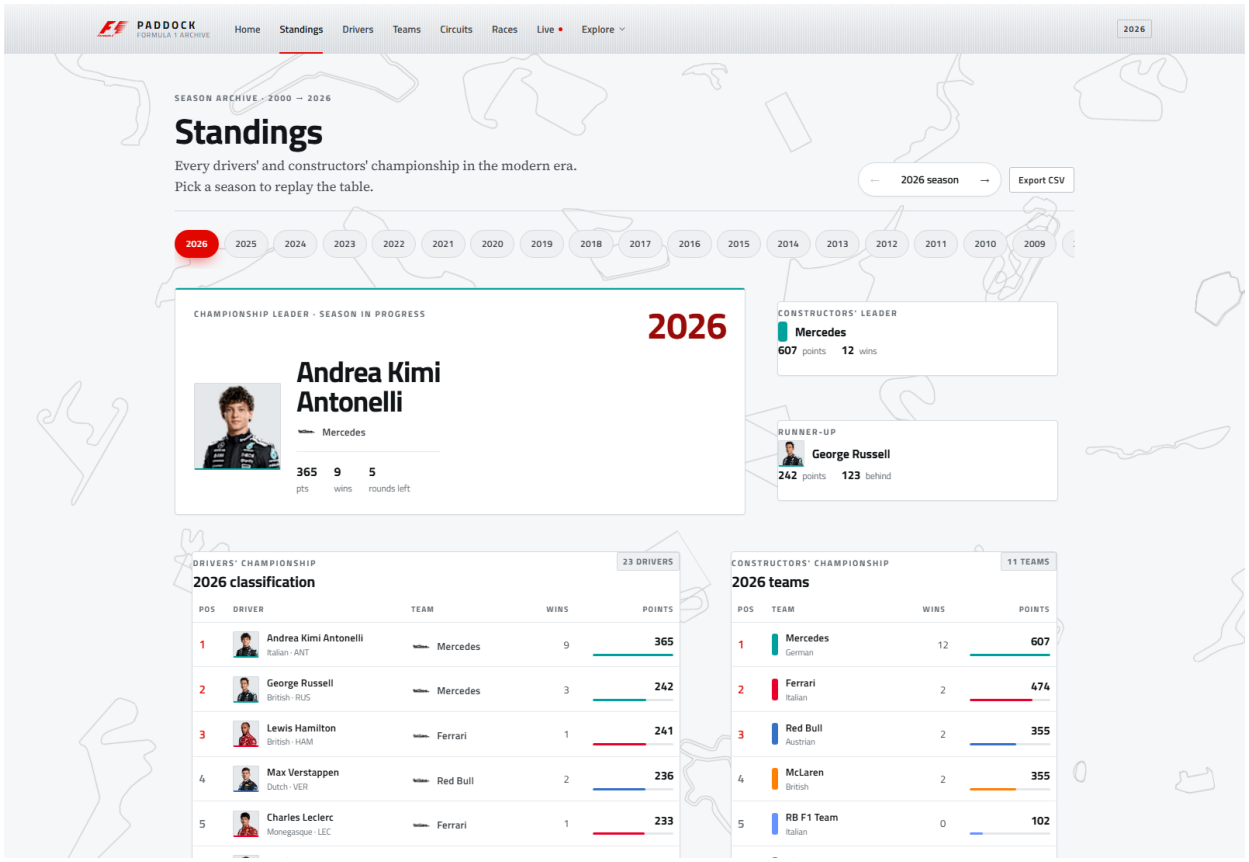


Figure 7 — Standings (/standings?year=2026). Drivers' and constructors' tables, the leader card, the points margin and the season switcher.

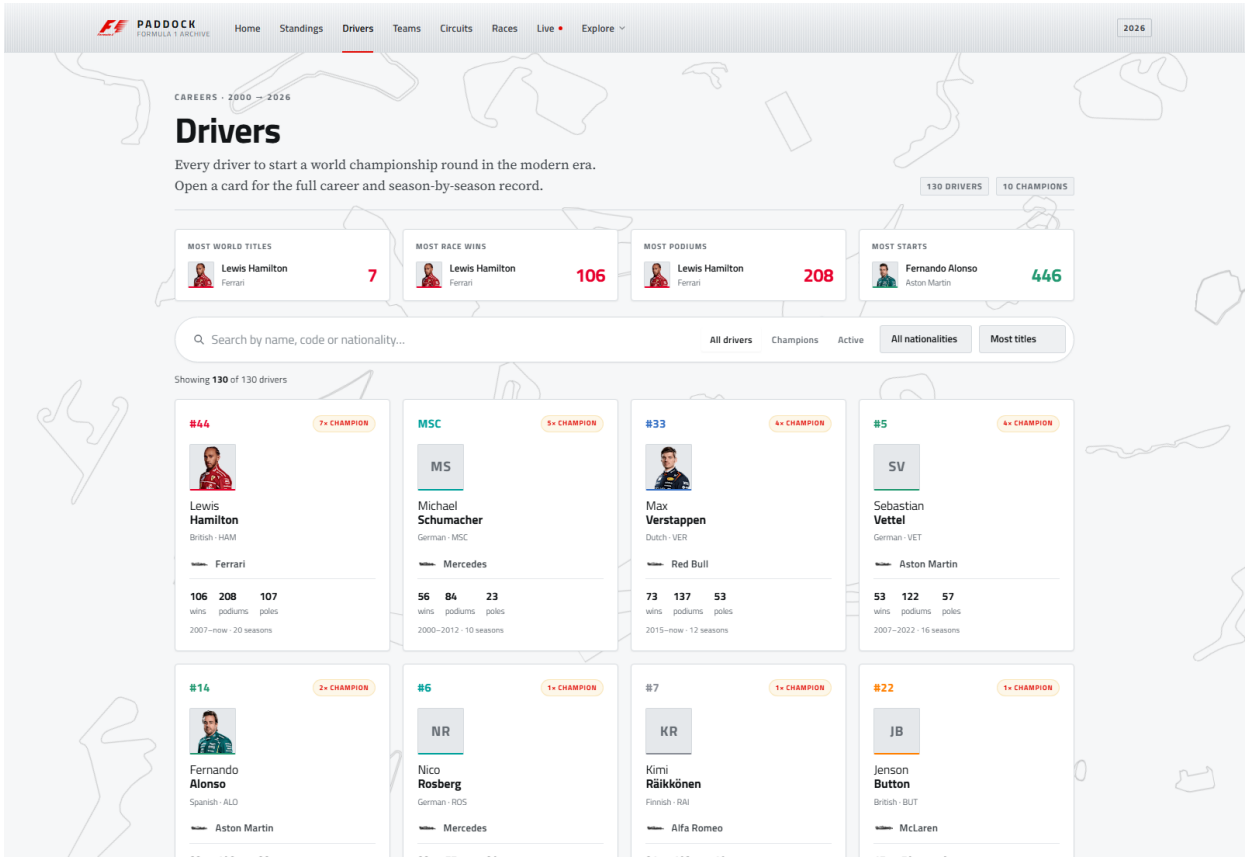


Figure 8 — Drivers (/drivers). Searchable, sortable grid. Cards open a career dialog with titles, seasons and teams.

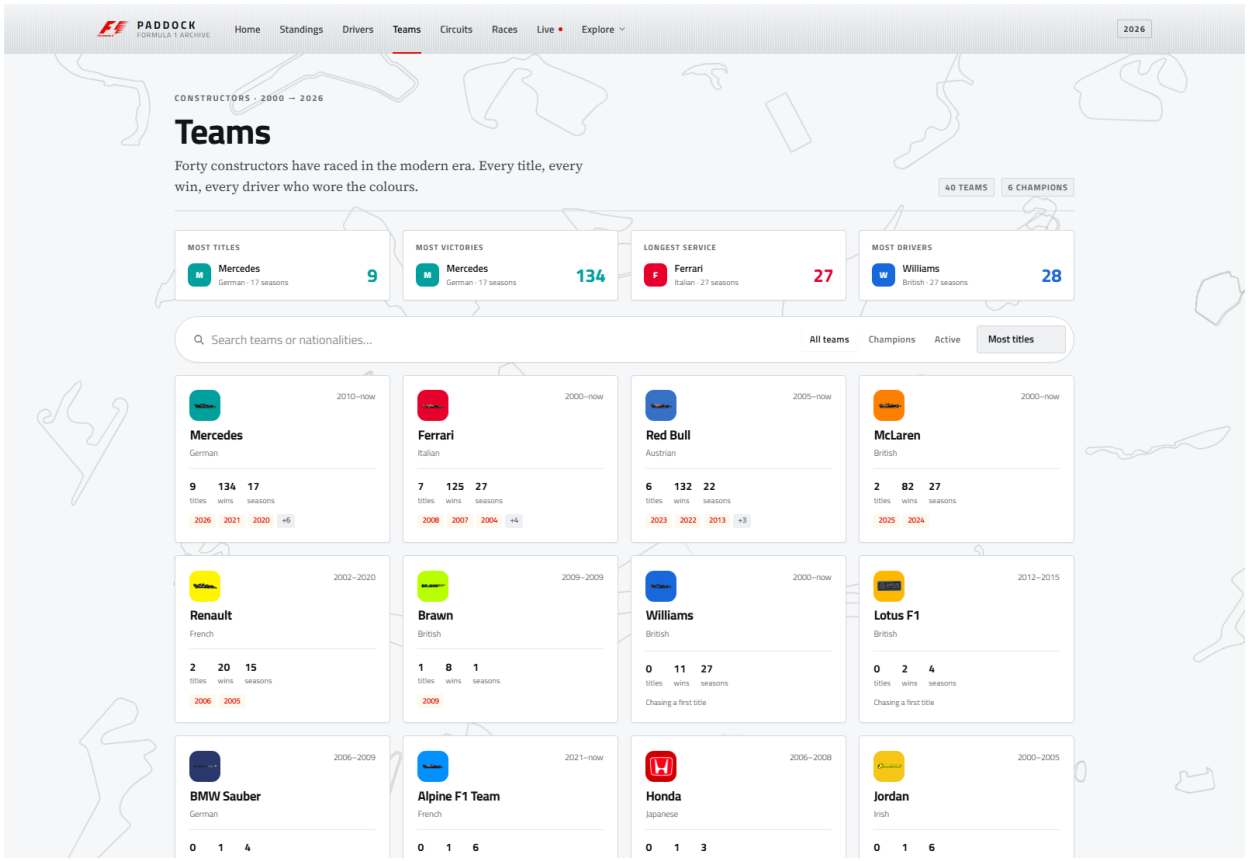


Figure 9 — Teams (/constructors). Real marks where available, typographic crests for the five unlicensed historical teams.

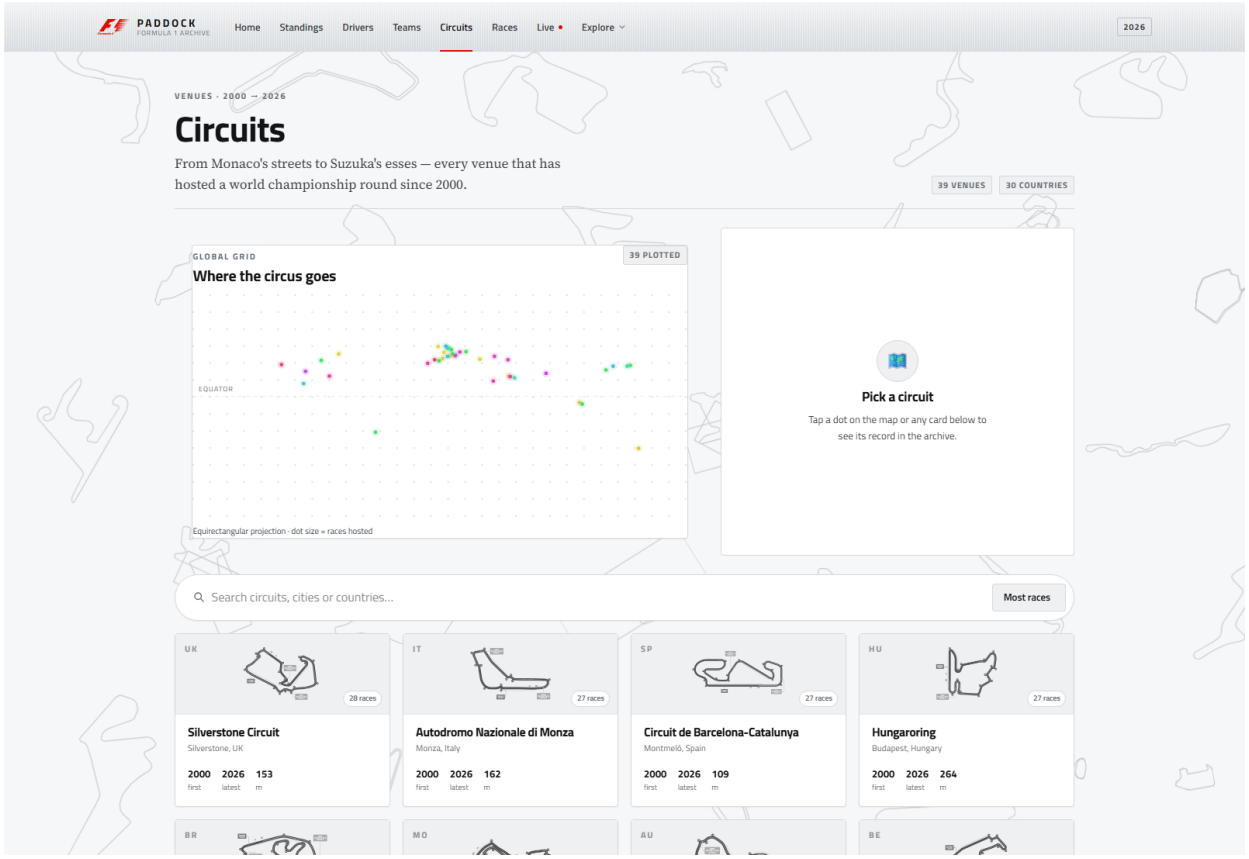


Figure 10 — Circuits (/circuits). World map, searchable library and detail panel, using the same traced outlines as the background.

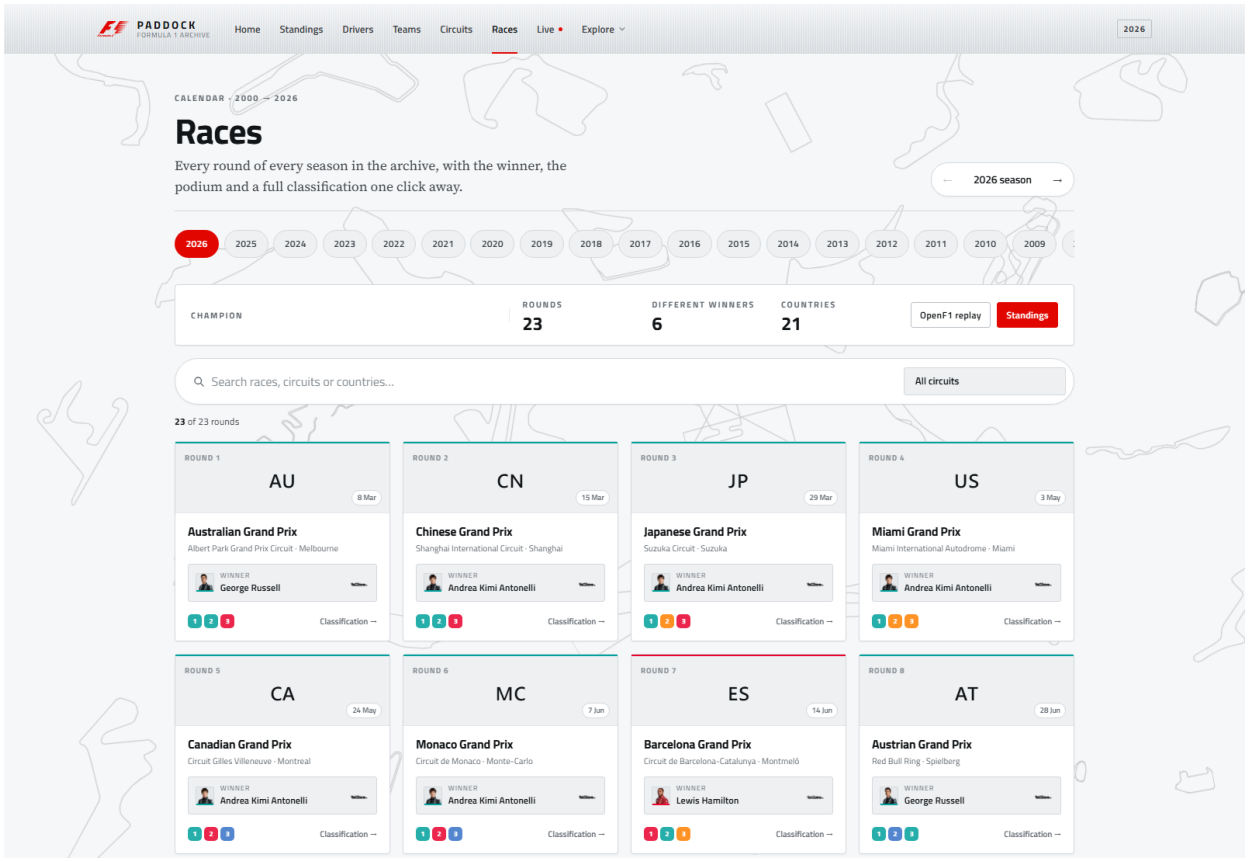


Figure 11 — Races (/races?year=2026). The 2026 calendar with winners and podiums where results exist, and a clear upcoming state where they do not.

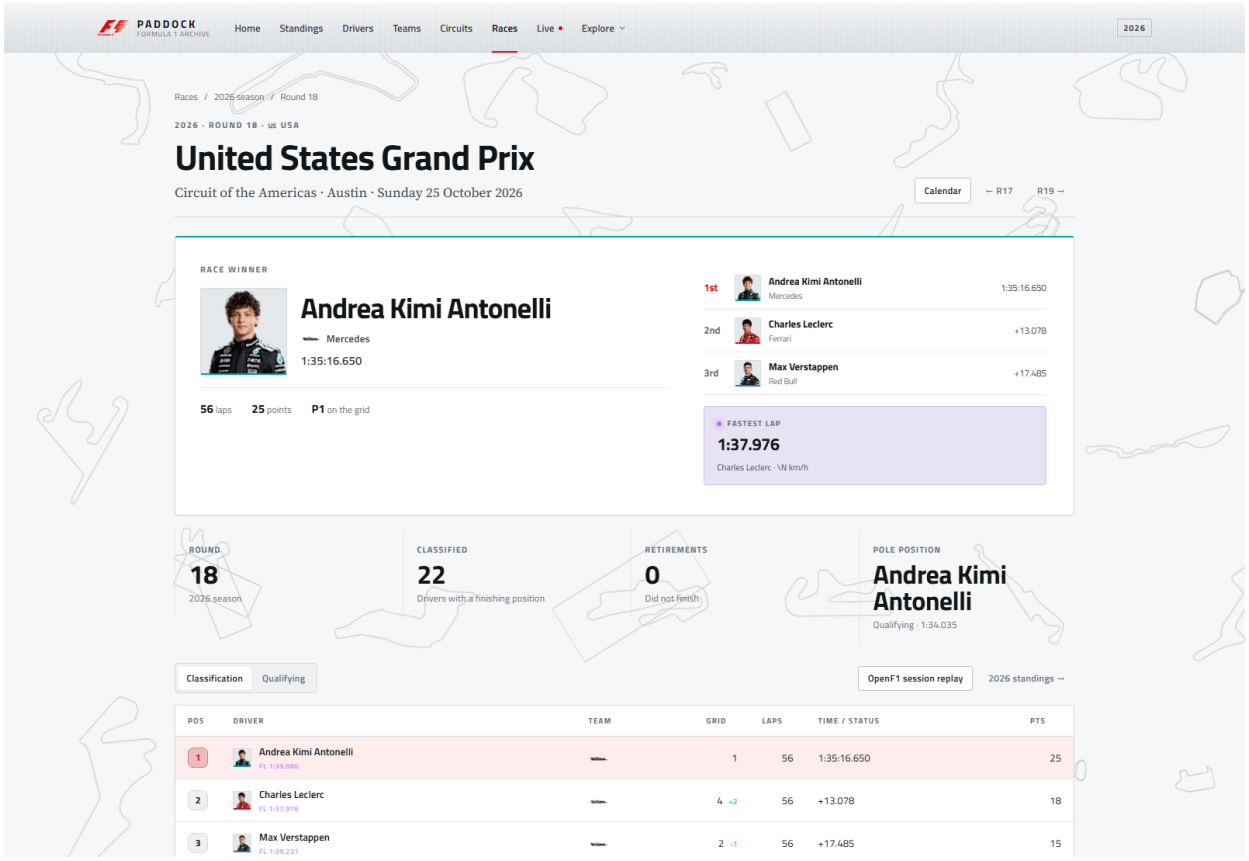


Figure 12 — Race detail (/races/202618). Winner hero, podium, fastest lap, classification, qualifying tabs and round navigation.

10.2 Live, long-form and credits

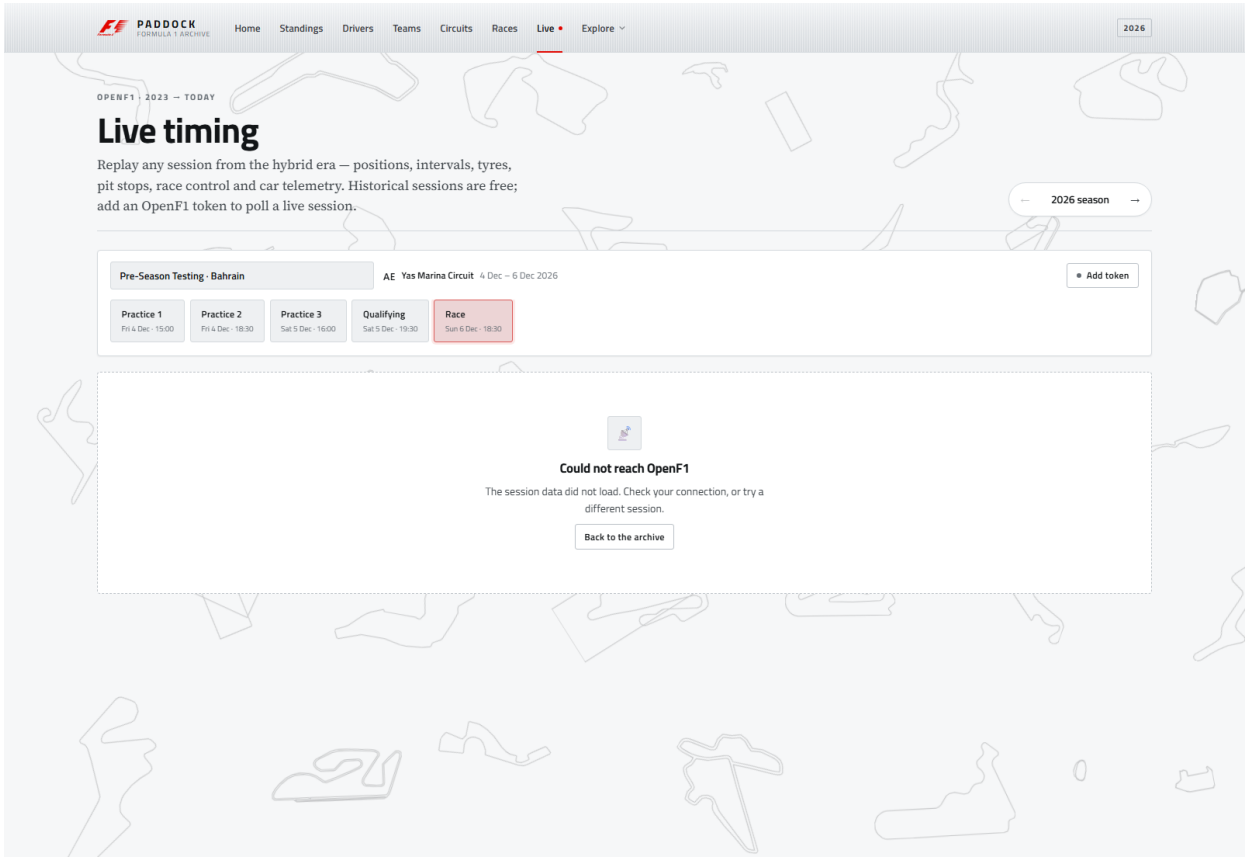


Figure 13 — Live timing (/live?year=2026). Session replay: timing tower, weather, race control and the per-lap telemetry picker.

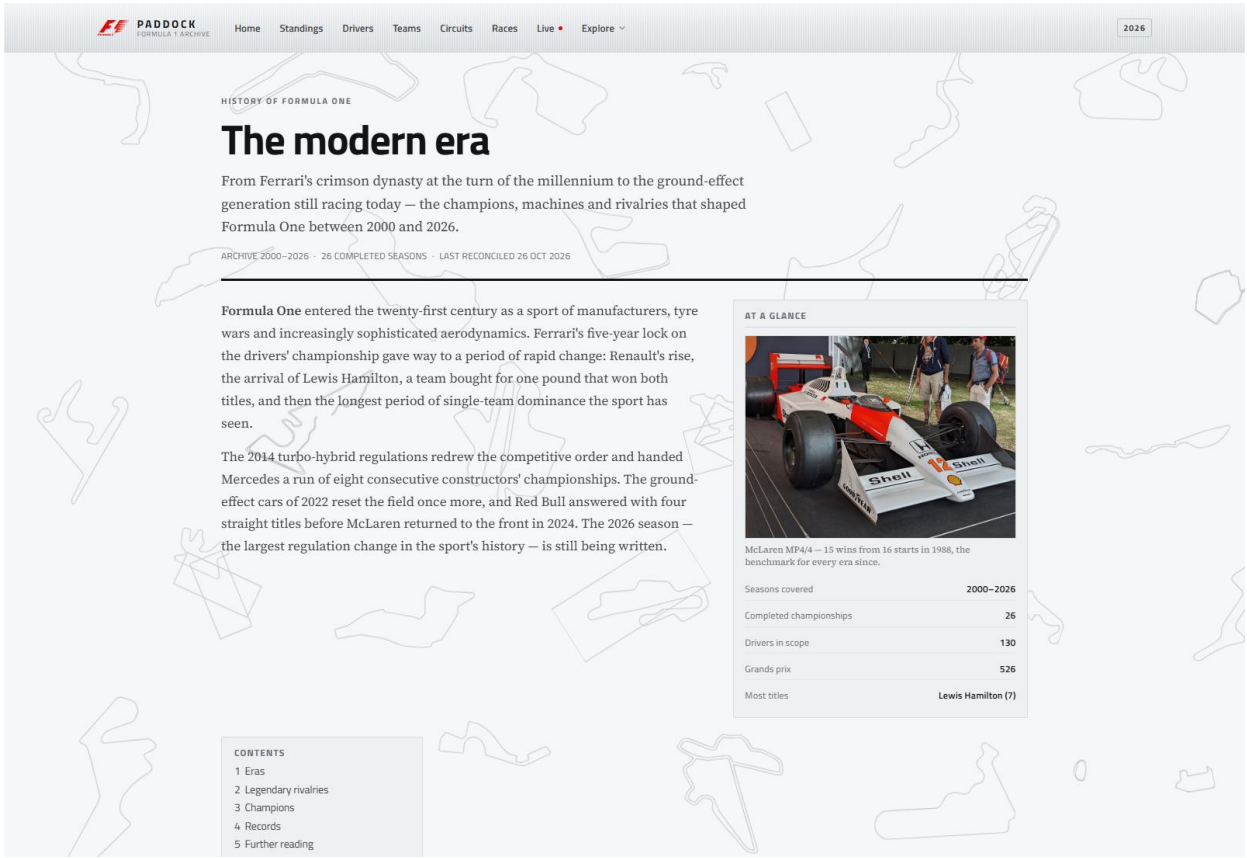


Figure 14 — History (/history). Long-form article with infobox, five eras, four rivalries, champions and records.

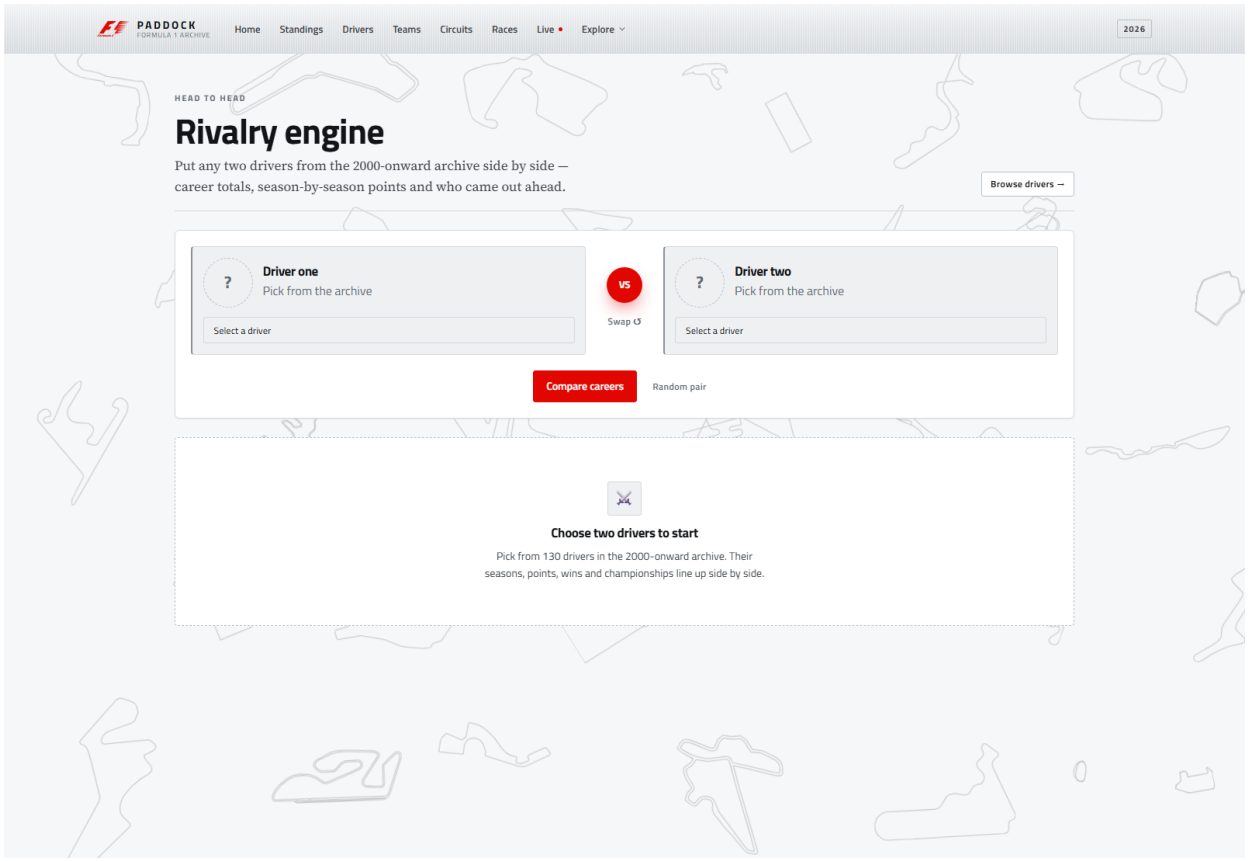


Figure 15 — Rivalry (/rivalry). Two-driver picker, career totals, points chart, season table and head-to-head score.

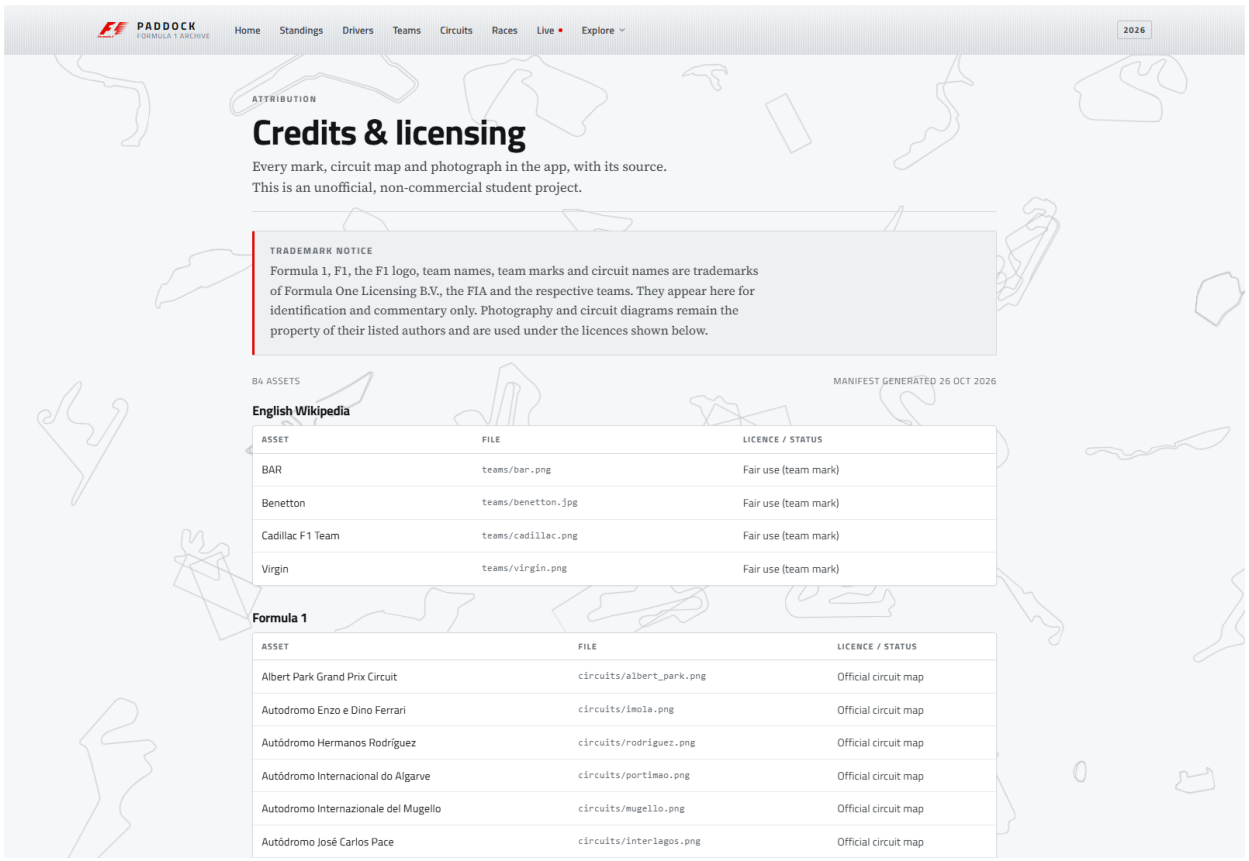


Figure 16 — Credits (/credits). Every image set with its source and licence, plus the trademark notice.

11. Performance and accessibility

A 1.3 MB dataset and a live API could easily produce a slow application. The build stays fast by parsing once, splitting by route and never downloading more telemetry than the reader asked for.

11.1 Bundle and loading

The production initial bundle is **580.38 kB raw / 154.23 kB transferred**. That includes the Angular runtime, the eager Home page and the shared component library. Every other route is a separate lazy chunk, with the live timing screen the largest and still under 45 kB. Angular's budgets are set to 1.5 MB warning and 2.5 MB error for the initial bundle, so the build has a wide margin before it fails.

- The archive is fetched once as ten parallel requests — nine CSVs and the manifest — and indexed in memory; pages never re-fetch a file.
- Derived views are cached with `shareReplay`, so navigating back to a page does not repeat the work.
- OpenF1 responses are cached by path and query; the token bucket prevents a burst of page interactions from exceeding the API limit.
- Telemetry requests are scoped to one driver and one lap window; whole-session car data is never downloaded.
- Images are local and lazy-loaded, with manifest checks that avoid 404s.
- The background is pure CSS transforms on inline SVG paths; there are no image requests and no JavaScript animation loop.

11.2 Accessibility

Accessibility was treated as a set of concrete interface decisions rather than a checklist added at the end:

- A skip link targets `#content`, and the main element is focusable.
- Headings follow a document outline; tables carry captions and header cells; tabs expose `aria-selected`.
- Controls have visible labels or accessible names, including icon-only buttons.
- Focus is never removed without a replacement; the focus ring is part of the token system.
- The mobile navigation is a keyboard-operable Material sheet, not a hover-only menu.
- `prefers-reduced-motion` disables the background loop; the content layer has no looping animation.
- Colour is not the only carrier of meaning: wins, positions and status also appear as text.

The project has not been through a formal WCAG 2.2 audit or a screen-reader test matrix. The claims above describe the implementation, not a certification. A proper audit — automated axe checks plus manual keyboard and screen-reader passes — is the obvious next step for the accessibility work.

11.3 Data-scale performance

Ten thousand results and ten thousand qualifying rows are small for a browser but large enough to punish a careless implementation. The service builds maps for drivers, constructors, circuits and races, then groups results and qualifying by race and by driver. Page code reads from those maps instead of filtering the full arrays. The cost is paid once, on the first subscription, and the loading signal gives the interface something honest to show while it happens.

The 2026 calendar is bundled in full, including races that have not run. That keeps the next-race card and the calendar complete without a second data source, at the cost of a small amount of empty data that the UI must handle explicitly. The complete/in-progress distinction in `meta.json` is what makes that safe.

12. Limitations and future work

The application is complete as a student project, but it is not finished as a product. The gaps below are known, documented and deliberately left visible.

12.1 Data limitations

- **2026 is in progress.** Eighteen of twenty-three rounds were complete on 26 October 2026. The app shows a leader, not a champion.
- **The 2026 calendar has unusual entries.** Sakhir and Jeddah were cancelled, Sepang hosts the Bahrain Grand Prix in Malaysia, and Madrid uses the `madring` circuit reference. The UI follows the data rather than assuming a conventional calendar.
- **Sprint results are not in the bundled archive.** Sprint weekends appear through their Grand Prix results and standings, but a separate sprint classification is not yet stored.
- **Portraits start in 2023.** OpenF1 lists drivers from 2023 onward, so earlier drivers use monograms. Arvid Lindblad has no portrait yet.
- **The archive is a snapshot.** It is reconciled to 26 October 2026 and must be refreshed with `npm run data:build` to move forward.

12.2 Technical limitations

- **No unit or end-to-end test suite.** Verification is build-based and route-based; there are no Angular TestBed specs or Playwright journeys committed.
- **No formal accessibility audit.** The implementation follows accessible patterns, but it has not been tested with a screen-reader matrix or an automated axe run.
- **True live polling needs a paid token.** The free tier is a replay service; the live indicator and 15-second refresh only activate with a bearer token.
- **One theme only.** There is no dark mode. That is a design decision, but it is also a limitation for readers who prefer one.
- **The optional backend is unmaintained relative to the frontend.** It still runs, but it is not part of the shipped data path and does not share the frontend's derived models.

12.3 Next steps

The most valuable next step is not a new screen; it is a test suite. A small set of unit tests around the archive derivations — complete seasons, latest race, rivalry scoring, cancelled rounds — would protect the parts most likely to break when the data is refreshed. A Playwright journey through the eleven routes would then cover the integration layer, and an axe pass would turn the accessibility claims into evidence.

After that, the data work is straightforward: add sprint classifications, widen portrait coverage where a licence allows, and add a server-side cache for OpenF1 so a shared deployment does not ask every visitor to hold a token. On the interface side, saved favourite drivers and deeper stint/tyre analysis are natural extensions of the existing models.

The design system is deliberately stable. Any future screen should reuse the panel, header, tile, badge and table primitives; the quickest way to damage the project would be to add a twelfth screen with its own surface language.

13. Conclusion

Paddock shows that a static Angular bundle can carry a serious archive and a live session viewer without a runtime server, and that a restrained visual system can make thirty decorative circuits feel like part of the subject rather than noise.

The project met its functional goals. Twenty-seven seasons are searchable and cross-referenced; eleven screens cover the archive, the live session, the history and the credits; and the live page degrades cleanly from a token-enabled poller to a historical replay. The production build is 580.38 kB raw and 154.23 kB transferred, with every non-home route split into its own chunk.

The more interesting result is methodological. Treating documentation as a build artefact — code listings sliced from source, counts read from `meta.json`, screenshots captured from the production bundle — removed a whole class of drift. The same habit changed the design work: the background was not accepted because it looked atmospheric, but because its ink centroid moved from 71/67 to 49/51 and its coverage could be measured at three widths.

There are real gaps: no test suite, no formal accessibility audit, no sprint classifications and no 2026 champion to name. Those are documented rather than papered over. For a semester project, the valuable outcome is not only the application but the repeatable process behind it: lock the data, build one complete screen at a time, measure the claims, and generate the paperwork from the same source tree.

References

Sources used for data, imagery, platform behaviour and accessibility guidance.

[1] **Ergast Motor Racing Data.** Historical Formula 1 dataset, 1950–2024. https://ergast.com/mrd/

[2] **Jolpica F1.** Community-maintained continuation of the Ergast API; source of the 2025–2026 data. https://github.com/jolpica/jolpica-f1

[3] **OpenF1.** Public Formula 1 session, timing, race control and telemetry API. https://openf1.org/

[4] **Angular.** Standalone components, signals, routing and the CLI build pipeline. https://angular.dev/

[5] **Angular Material.** Dialog, navigation sheet and accessibility primitives. https://material.angular.io/

[6] **PapaParse / ngx-papaparse.** Streaming CSV parsing used for the bundled archive. https://www.papaparse.com/

[7] **Formula 1 media CDN.** Team marks and official circuit map imagery, used for identification. https://www.formula1.com/

[8] **Wikimedia Commons.** Archive photography under Creative Commons or public-domain licences. https://commons.wikimedia.org/

[9] **MDN — prefers-reduced-motion.** CSS media feature used to disable the background animation. https://developer.mozilla.org/en-US/docs/Web/CSS/@media/prefers-reduced-motion

[10] **W3C — WCAG 2.2.** Accessibility guidance referenced by the interface decisions. https://www.w3.org/TR/WCAG22/

Appendix A — Source map

Every file below is part of the shipped frontend. Line counts are read from the working tree when this report is generated, so the map describes the same revision as the listings.

Table 14 — Source map of the shipped frontend

LAYER	FILE	LINES	RESPONSIBILITY
Shell	app.ts	161	Skip link, background, navbar, router outlet and footer
Shell	app.config.ts	27	Router, HTTP, animations, Material dialog providers
Shell	app.routes.ts	64	Eleven routes, lazy loading, timeline redirect and wildcard
Shell	components/navbar/navbar.component.ts	472	Desktop navigation, active rule and mobile sheet
Config	core/config/data.config.ts	79	Archive window, live start year, team colours
Config	core/config/asset-manifest.ts	226	Generated list of images that exist, with credits
Config	core/config/circuit-outlines.ts	171	Generated SVG paths for the circuit background
Models	core/models/driver.model.ts	246	Archive entities and derived career/standings types
Models	core/models/openf1.model.ts	209	OpenF1 session, timing, telemetry and weather types
Services	core/services/csv-data.service.ts	824	Archive load, indexes, careers, standings, summaries, rivalry
Services	core/services/openf1.service.ts	343	Cached API client, token bucket, live board, telemetry
Utilities	core/utils/format.ts	90	Number, date, gap and colour formatting
Utilities	core/utils/country.ts	70	Country names, flags and nationality normalisation
Shared	shared/components/panel	93	The matte silver surface used by every page
Shared	shared/components/page-header	85	Eyebrow, title and description rhythm
Shared	shared/components/stat-tile	80	Number and label tile
Shared	shared/components/driver-avatar	103	Manifest-backed portrait with monogram fallback
Shared	shared/components/team-badge	106	Manifest-backed logo with typographic crest fallback
Shared	shared/components/spark-chart	159	Hand-built SVG trend line
Shared	shared/components/telemetry-chart	190	Speed, gear and DRS trace for one lap
Shared	shared/components/skeleton-loader	37	Loading placeholders that match the final layout
Shared	shared/components/empty-state	64	Designed absence state
Shared	shared/components/track-background	259	Thirty-map full-bleed field and responsive rebuild
Pages	pages/home	895	MP4/4 hero, top five, totals, latest and next race
Pages	pages/standings	854	Drivers' and constructors' tables, leader card, export
Pages	pages/drivers	604	Searchable, sortable driver grid

LAYER	FILE	LINES	RESPONSIBILITY
Pages	pages/drivers/driver-profile-dialog	431	Career dialog with titles, seasons and teams
Pages	pages/constructors	757	Team grid, crests, title years and line-ups
Pages	pages/circuits	740	Map, searchable library and circuit detail
Pages	pages/races	791	Season calendar, winners, podiums and year switcher
Pages	pages/race-detail	924	Winner, podium, classification, qualifying and navigation
Pages	pages/live	1529	Session replay, timing tower, weather, race control, telemetry
Pages	pages/history	1084	Long-form article, eras, rivalries, champions and records
Pages	pages/credits	158	Generated image credits and trademark notice
Feature	features/rivalry	794	Two-driver comparison, chart and season table
Styles	styles/_tokens.scss	103	Colour, radius, elevation and motion tokens
Styles	styles.scss	739	Global silver surfaces, buttons, tables and accessibility utilities

Appendix B — Build and data scripts

The repository is reproducible from these commands. The data and asset scripts are Node; the three document builders are Python.

Table 15 — Scripts and their outputs

COMMAND	SCRIPT	WHAT IT PRODUCES
`npm run data:prune`	`scripts/prune-archive.mjs`	Rebuilds the 2000–2024 base CSVs and season standings
`npm run data:sync`	`scripts/sync-current-season.mjs`	Appends 2025–2026 from Jolpica and rewrites `meta.json`
`npm run data:build`	Both data scripts	A full archive refresh in the correct order
`npm run assets:fetch`	`scripts/fetch-assets.mjs`	Downloads logos, maps, portraits and photos; writes the manifest
`npm run outlines:build`	`scripts/extract-circuit-outlines.mjs`	Traces circuit maps into stroke-only SVG paths
`npm run build`	Angular CLI 21	Production bundle in `frontend/dist/frontend/browser`
`npm run docs:build`	`scripts/build-docs.py`	`README.docx` with cover, contents and screenshots
`npm run deck:build`	`scripts/build-deck.py`	`f1-presentation.pptx`, 15 slides
`python scripts/build-report.py`	`scripts/build-report.py`	This report, with generated diagrams, listings and screenshots

Appendix C — Rebuilding the project

The sequence below rebuilds the data, the application and the documents from a clean checkout. The PDF exports use Microsoft Office because the table of contents and slide notes must be refreshed by the same engine that lays them out.

Listing 13 — Full rebuild sequence · powershell

```
# 1. data and assets
npm run data:build
npm run assets:fetch
npm run outlines:build


# 2. application
cd frontend
npm install
npm run build
npx http-server dist/frontend/browser -p 8080 -c-1


# 3. documents
cd ..
npm run docs:build
npm run deck:build
python scripts/build-report.py


# 4. PDF export (Word / PowerPoint COM)
#    open the DOCX, update fields and the TOC, then SaveAs PDF (17)
#    open the PPTX and SaveAs PDF (32)
```

The screenshots in section 10 are captured from the production preview, not from the development server, so they show the bundle that would actually be deployed. Re-capturing them is the last step before rebuilding the documents; otherwise the report and the deck describe an older interface.